

```
"""
```

historyentk_bot – полный production-ready Telegram-бот для тестирования абитуриентов

Структура файла:

CONFIG	– токены, константы, ENV
DATABASE	– SQLite через aiosqlite, все таблицы
STATES	– FSM состояния
UTILS	– вспомогательные функции
ANTI_SPAM	– rate limit, cooldown
QUIZ SYSTEM	– вопросы, темы, сессии, spaced repetition
ECONOMY	– монеты, XP, уровни, streak
PREMIUM	– подписки, лимиты, антифарм
REFERRALS	– реферальная система
ACHIEVEMENTS	– достижения
NOTIFICATIONS	– push, авто-пауза, reminder
IMPORT SYSTEM	– текст, Quiz Poll, файлы
ADMIN PANEL	– инлайн-админка
CALLBACKS	– все callback роутеры
HANDLERS	– message handlers
MAIN	– запуск

```
"""
```

```
# |  
# | IMPORTS |  
# |
```

```
import asyncio  
import logging  
import os  
import re  
import json  
import random  
import string  
import time  
import math  
  
from datetime import datetime, timedelta, timezone  
from html import escape  
from itertools import count  
from typing import Final, Optional, Any  
  
import aiosqlite  
  
from telegram import (  
    Update,
```

```

InlineKeyboardMarkup,
InlineKeyboardButton,
ReplyKeyboardMarkup,
ReplyKeyboardRemove,
KeyboardButton,
)
from telegram.constants import ParseMode, ChatMemberStatus
from telegram.error import Forbidden, BadRequest, TelegramError
from telegram.ext import (
    ApplicationBuilder,
    CommandHandler,
    MessageHandler,
    CallbackQueryHandler,
    PollAnswerHandler,
    ConversationHandler,
    ContextTypes,
    filters,
    JobQueue,
)

# ===== CONFIG =====
# | CONFIG |
# =====

BOT_TOKEN: str = os.getenv("BOT_TOKEN", "").strip()
ADMINS_RAW: str = os.getenv("ADMINS", "").strip()
CHANNEL_ID: str = os.getenv("CHANNEL_ID", "").strip() # @channel или -100xxx
DB_PATH: str = os.getenv("DB_PATH", "bot.db")

if not BOT_TOKEN:
    raise ValueError("Не найден BOT_TOKEN в переменных окружения")
if not ADMINS_RAW:
    raise ValueError("Не найден ADMINS в переменных окружения")

try:
    ADMINS: Final[set[int]] = {
        int(a.strip()) for a in ADMINS_RAW.split(",") if a.strip()
    }
except ValueError as exc:
    raise ValueError("ADMINS должен содержать только числовые Telegram ID") from exc

# — Логирование —
logging.basicConfig(
    format="%(asctime)s | %(name)s | %(levelname)s | %(message)s",
    level=logging.INFO,
)

logger = logging.getLogger( name )
```

```

# — Уровни пользователей —————
LEVELS = [
    (0,      "🌱 Новичок"),
    (500,    "📖 Абитуриент"),
    (1500,   "🎓 Ученик"),
    (3500,   "🧠 Знаток"),
    (7000,   "👤 Эксперт"),
    (13000,  "⭐ Мастер ЕНТ"),
    (25000,  "🏆 Легенда"),
]

# — Сложность вопросов → монеты / XP —————
DIFFICULTY_REWARDS = {
    "easy":   {"coins": 1, "xp": 8},
    "medium": {"coins": 2, "xp": 12},
    "hard":   {"coins": 3, "xp": 18},
    "rare":   {"coins": 5, "xp": 25},
}

# — Лимиты по умолчанию —————
DEFAULT_DAILY_LIMIT_FREE      = 55
DEFAULT_DAILY_LIMIT_PREMIUM   = 9999 # безлимит
DEFAULT_COIN_LIMIT_FREE       = 55   # монет в день (free)
DEFAULT_COIN_LIMIT_PREMIUM    = 100   # монет за вопросы (premium), рефералы — отд

# — Premium за монеты —————
PREMIUM_COIN_TIERS = {
    "3d": {"days": 3, "coins": 800, "label": "3 дня"},
    "7d": {"days": 7, "coins": 1800, "label": "7 дней"},
}

PREMIUM_COIN_COOLDOWN_DAYS = 30 # раз в 30 дней можно купить premium за монеты

# — Streak bonuses —————
STREAK_BONUSES = {5: 10, 10: 25, 30: 75, 60: 150}

# — Spaced repetition приоритеты —————
# wrong_streak → через сколько вопросов повторить
REPEAT_AFTER = {0: 5, 1: 8, 2: 15, 3: 30, 4: 999}

# — Ин-мемори для старого функционала (A/B/C/D) —————
from itertools import count as _count
REQUESTS: dict[int, dict] = {}
RESPONSES: dict[int, dict] = {}
REQUEST_SEQ = _count(1001)
RESPONSE_SEQ = _count(5001)
BLOCKED_USERS: set[int] = set()

```



```

        notified_expire INTEGER NOT NULL DEFAULT 0,
        FOREIGN KEY(user_id) REFERENCES users(id)
    );

-- Темы
CREATE TABLE IF NOT EXISTS topics (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT NOT NULL,
    description    TEXT,
    is_premium    INTEGER NOT NULL DEFAULT 0,
    is_active     INTEGER NOT NULL DEFAULT 1,
    created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

-- Вопросы
CREATE TABLE IF NOT EXISTS questions (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    qid           TEXT NOT NULL UNIQUE,    -- Q-XXXXXX
    topic_id      INTEGER NOT NULL,
    question_text TEXT NOT NULL,
    option_a      TEXT NOT NULL,
    option_b      TEXT NOT NULL,
    option_c      TEXT NOT NULL,
    option_d      TEXT NOT NULL,
    correct_index INTEGER NOT NULL,    -- 0-3
    difficulty    TEXT NOT NULL DEFAULT 'medium',
    explanation   TEXT,
    is_rare       INTEGER NOT NULL DEFAULT 0,
    is_active     INTEGER NOT NULL DEFAULT 1,
    created_at    TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY(topic_id) REFERENCES topics(id)
);

-- Ответы пользователей (история)
CREATE TABLE IF NOT EXISTS user_answers (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER NOT NULL,
    question_id   INTEGER NOT NULL,
    is_correct    INTEGER NOT NULL,
    answered_at   TEXT NOT NULL DEFAULT (datetime('now')),
    wrong_streak  INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY(user_id) REFERENCES users(id),
    FOREIGN KEY(question_id) REFERENCES questions(id)
);

-- Сессии тестирования
CREATE TABLE IF NOT EXISTS sessions (

```

```

    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER NOT NULL UNIQUE,
    topic_id      INTEGER,
    question_id   INTEGER,
    poll_message_id INTEGER,
    chat_id       INTEGER,
    correct_count INTEGER NOT NULL DEFAULT 0,
    wrong_count   INTEGER NOT NULL DEFAULT 0,
    total_asked   INTEGER NOT NULL DEFAULT 0,
    daily_count   INTEGER NOT NULL DEFAULT 0,
    coins_earned  INTEGER NOT NULL DEFAULT 0,
    xp_earned     INTEGER NOT NULL DEFAULT 0,
    session_date  TEXT NOT NULL DEFAULT (date('now')),
    status        TEXT NOT NULL DEFAULT 'active', -- active/paused/fin
    missed_polls  INTEGER NOT NULL DEFAULT 0,
    reminder_sent INTEGER NOT NULL DEFAULT 0,
    paused_at     TEXT,
    updated_at    TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY(user_id) REFERENCES users(id),
    FOREIGN KEY(topic_id) REFERENCES topics(id),
    FOREIGN KEY(question_id) REFERENCES questions(id)
);

```

-- Рефералы

```

CREATE TABLE IF NOT EXISTS referrals (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    referrer_id   INTEGER NOT NULL,
    referred_id   INTEGER NOT NULL UNIQUE,
    coins_given   INTEGER NOT NULL DEFAULT 0,
    created_at    TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY(referrer_id) REFERENCES users(id),
    FOREIGN KEY(referred_id) REFERENCES users(id)
);

```

-- Достижения

```

CREATE TABLE IF NOT EXISTS achievements (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER NOT NULL,
    code         TEXT NOT NULL,
    earned_at     TEXT NOT NULL DEFAULT (datetime('now')),
    UNIQUE(user_id, code),
    FOREIGN KEY(user_id) REFERENCES users(id)
);

```

-- Апелляции

```

CREATE TABLE IF NOT EXISTS appeals (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        user_id            INTEGER NOT NULL,
        question_id        INTEGER NOT NULL,
        comment            TEXT,
        source_ref         TEXT,
        status             TEXT NOT NULL DEFAULT 'pending', -- pending/accepted
        admin_note         TEXT,
        created_at         TEXT NOT NULL DEFAULT (datetime('now')),
        FOREIGN KEY(user_id) REFERENCES users(id),
        FOREIGN KEY(question_id) REFERENCES questions(id)
    );

```

-- Черновики импорта

```

CREATE TABLE IF NOT EXISTS drafts (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    admin_id      INTEGER NOT NULL,
    data          TEXT NOT NULL, -- JSON
    created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

```

-- Настройки бота

```

CREATE TABLE IF NOT EXISTS settings (
    key          TEXT PRIMARY KEY,
    value        TEXT NOT NULL
);

```

-- Логи транзакций монет

```

CREATE TABLE IF NOT EXISTS coin_log (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id       INTEGER NOT NULL,
    delta         INTEGER NOT NULL,
    reason        TEXT NOT NULL,
    created_at    TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY(user_id) REFERENCES users(id)
);

```

-- Закрытые доступы

```

CREATE TABLE IF NOT EXISTS closed_access (
    user_id       INTEGER PRIMARY KEY,
    topics        TEXT, -- JSON список topic_id или NULL (все)
    expires_at    TEXT,
    granted_by    INTEGER,
    FOREIGN KEY(user_id) REFERENCES users(id)
);

```

-- Покупки Premium за монеты (для cooldown)

```

CREATE TABLE IF NOT EXISTS premium_coin_purchases (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

        user_id      INTEGER NOT NULL,
        purchased_at TEXT NOT NULL DEFAULT (datetime('now')),
        days         INTEGER NOT NULL,
        coins_spent  INTEGER NOT NULL,
        FOREIGN KEY(user_id) REFERENCES users(id)
    );
"""

```

Дефолтные настройки

```

defaults = {
    "daily_limit_free":      str(DEFAULT_DAILY_LIMIT_FREE),
    "daily_limit_premium":   str(DEFAULT_DAILY_LIMIT_PREMIUM),
    "coin_limit_free":       str(DEFAULT_COIN_LIMIT_FREE),
    "coin_limit_premium":    str(DEFAULT_COIN_LIMIT_PREMIUM),
    "channel_id":            CHANNEL_ID,
    "premium_coins_enabled": "1",
    "premium_coin_3d":       "800",
    "premium_coin_7d":       "1800",
    "premium_coin_cooldown": "30",
    "coin_per_referral":     "200",
    "referral_limit_per_day": "5",
    "reminder_delay_minutes": "30",
    "antifarm_enabled":      "1",
    "x2_coins_active":       "0",
    "closed_mode":           "0",
}

for k, v in defaults.items():
    await db.execute(
        "INSERT OR IGNORE INTO settings(key, value) VALUES(?,?)", (k, v)
    )
await db.commit()

```

```

async def db_get_setting(key: str, default: str = "") -> str:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT value FROM settings WHERE key=?", (key,)) a
            row = await cur.fetchone()
    return row[0] if row else default

```

```

async def db_set_setting(key: str, value: str):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            "INSERT OR REPLACE INTO settings(key,value) VALUES(?,?)", (key, value)
        )
    await db.commit()

```

```

async def db_get_or_create_user(user) -> dict:
    """Получает или создаёт запись пользователя, возвращает dict."""
    ref_code = "".join(random.choices(string.ascii_uppercase + string.digits, k=8))
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("""
            INSERT OR IGNORE INTO users(id, username, first_name, last_name, ref_
            VALUES(?,?,?,?,?,datetime('now'))
            """, (user.id, user.username, user.first_name, user.last_name, ref_code))
        await db.execute("""
            UPDATE users SET username=?, first_name=?, last_name=?, last_active=d
            WHERE id=?
            """, (user.username, user.first_name, user.last_name, user.id))
        await db.commit()
        async with db.execute("SELECT * FROM users WHERE id=?", (user.id,)) as cu
            row = await cur.fetchone()
            cols = [d[0] for d in cur.description]
        return dict(zip(cols, row))

```

```

async def db_get_user(user_id: int) -> Optional[dict]:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT * FROM users WHERE id=?", (user_id,)) as cu
            row = await cur.fetchone()
            if not row:
                return None
            cols = [d[0] for d in cur.description]
        return dict(zip(cols, row))

```

```

async def db_update_user(user_id: int, **kwargs):
    if not kwargs:
        return
    sets = ", ".join(f"{k}=?" for k in kwargs)
    vals = list(kwargs.values()) + [user_id]
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(f"UPDATE users SET {sets} WHERE id=?", vals)
        await db.commit()

```

```

async def db_add_coins(user_id: int, delta: int, reason: str):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("UPDATE users SET coins=MAX(0,coins+?) WHERE id=?", (del
        await db.execute(
            "INSERT INTO coin_log(user_id,delta,reason) VALUES(?,?,?)",
            (user_id, delta, reason)
        )

```

```
await db.commit()
```

```
async def db_add_xp(user_id: int, xp: int) -> int:
    """Добавляет XP, возвращает новый уровень (или 0 если не изменился)."""
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("UPDATE users SET xp=xp+? WHERE id=?", (xp, user_id))
        await db.commit()
        async with db.execute("SELECT xp, level FROM users WHERE id=?", (user_id,
            row = await cur.fetchone()
    if not row:
        return 0
    total_xp, old_level = row
    new_level = 0
    for i, (req_xp, _) in enumerate(LEVELS):
        if total_xp >= req_xp:
            new_level = i
    if new_level != old_level:
        async with aiosqlite.connect(DB_PATH) as db:
            await db.execute("UPDATE users SET level=? WHERE id=?", (new_level, u
            await db.commit()
        return new_level
    return 0
```

```
async def db_is_premium(user_id: int) -> bool:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT expires_at FROM premium WHERE user_id=?", (user_id,)
        ) as cur:
            row = await cur.fetchone()
    if not row:
        return False
    try:
        exp = datetime.fromisoformat(row[0])
        return exp > datetime.now(timezone.utc).replace(tzinfo=None)
    except Exception:
        return False
```

```
async def db_grant_premium(user_id: int, days: int, source: str = "admin"):
    expires = datetime.utcnow() + timedelta(days=days)
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("""
            INSERT INTO premium(user_id, expires_at, source)
            VALUES(?,?,?)
            ON CONFLICT(user_id) DO UPDATE
```

```

        SET expires_at=MAX(expires_at, excluded.expires_at),
        source=excluded.source,
        notified_expire=0
    """", (user_id, expires.isoformat(), source))
    await db.commit()

```

```

async def db_revoke_premium(user_id: int):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("DELETE FROM premium WHERE user_id=?", (user_id,))
        await db.commit()

```

```

async def db_get_session(user_id: int) -> Optional[dict]:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT * FROM sessions WHERE user_id=?", (user_id,
            row = await cur.fetchone()
            if not row:
                return None
            cols = [d[0] for d in cur.description]
            return dict(zip(cols, row))

```

```

async def db_upsert_session(user_id: int, **kwargs):
    """Создаёт или обновляет сессию."""
    kwargs["updated_at"] = datetime.utcnow().isoformat()
    async with aiosqlite.connect(DB_PATH) as db:
        existing = await db.execute("SELECT id FROM sessions WHERE user_id=?", (u
        row = await existing.fetchone()
        if row:
            sets = ", ".join(f"{k}=?" for k in kwargs)
            vals = list(kwargs.values()) + [user_id]
            await db.execute(f"UPDATE sessions SET {sets} WHERE user_id=?", vals)
        else:
            kwargs["user_id"] = user_id
            cols = ", ".join(kwargs.keys())
            placeholders = ", ".join("?" for _ in kwargs)
            await db.execute(
                f"INSERT INTO sessions({cols}) VALUES({placeholders})",
                list(kwargs.values())
            )
        await db.commit()

```

```

async def db_get_topics(premium_ok: bool = False) -> list[dict]:
    async with aiosqlite.connect(DB_PATH) as db:
        if premium_ok:

```

```

        async with db.execute(
            "SELECT * FROM topics WHERE is_active=1 ORDER BY name", ()
        ) as cur:
            rows = await cur.fetchall()
            cols = [d[0] for d in cur.description]
    else:
        async with db.execute(
            "SELECT * FROM topics WHERE is_active=1 AND is_premium=0 ORDER BY
        ) as cur:
            rows = await cur.fetchall()
            cols = [d[0] for d in cur.description]
    return [dict(zip(cols, r)) for r in rows]

async def db_count_questions(topic_id: int) -> int:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT COUNT(*) FROM questions WHERE topic_id=? AND is_active=1", (t
        ) as cur:
            row = await cur.fetchone()
    return row[0] if row else 0

async def db_next_question(user_id: int, topic_id: int, session: dict) -> Optiona
"""
Выбирает следующий вопрос с учётом spaced repetition и миксования.
80% новые / ещё не отвеченные, 20% старые. Ошибочные — приоритет.
"""
    async with aiosqlite.connect(DB_PATH) as db:
        # Вопросы с ошибками (pending repeat)
        async with db.execute("""
            SELECT q.*, ua.wrong_streak
            FROM questions q
            JOIN user_answers ua ON ua.question_id=q.id
            WHERE q.topic_id=? AND q.is_active=1 AND ua.user_id=?
            AND ua.is_correct=0 AND ua.wrong_streak>0
            ORDER BY ua.wrong_streak DESC, RANDOM()
            LIMIT 10
            """, (topic_id, user_id)) as cur:
            wrong_rows = await cur.fetchall()
            wrong_cols = [d[0] for d in cur.description]
            wrong_qs = [dict(zip(wrong_cols, r)) for r in wrong_rows]

        # Уже отвеченные правильно (для повтора)
        async with db.execute("""
            SELECT q.*
            FROM questions q

```

```

        JOIN user_answers ua ON ua.question_id=q.id
        WHERE q.topic_id=? AND q.is_active=1 AND ua.user_id=? AND ua.is_corre
        ORDER BY RANDOM()
        LIMIT 5
    """
    (topic_id, user_id)) as cur:
        done_rows = await cur.fetchall()
        done_cols = [d[0] for d in cur.description]
        done_qs = [dict(zip(done_cols, r)) for r in done_rows]

    # Новые вопросы (ни разу не отвечал)
    async with db.execute("""
        SELECT q.* FROM questions q
        WHERE q.topic_id=? AND q.is_active=1
        AND q.id NOT IN (
            SELECT question_id FROM user_answers WHERE user_id=?
        )
        ORDER BY RANDOM()
        LIMIT 20
    """, (topic_id, user_id)) as cur:
        new_rows = await cur.fetchall()
        new_cols = [d[0] for d in cur.description]
        new_qs = [dict(zip(new_cols, r)) for r in new_rows]

    # Выбор по приоритету
    if wrong_qs and random.random() < 0.6:
        return wrong_qs[0]
    if new_qs and random.random() < 0.8:
        return random.choice(new_qs[:10])
    if done_qs and random.random() < 0.2:
        return random.choice(done_qs)
    if new_qs:
        return random.choice(new_qs)
    if wrong_qs:
        return wrong_qs[0]
    if done_qs:
        return random.choice(done_qs)
    return None

async def db_record_answer(user_id: int, question_id: int, is_correct: bool):
    """Записывает ответ, обновляет wrong_streak для spaced repetition."""
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT id, wrong_streak FROM user_answers WHERE user_id=? AND questi
            (user_id, question_id)
        ) as cur:
            existing = await cur.fetchone()

```

```

    if existing:
        old_streak = existing[1]
        new_streak = 0 if is_correct else old_streak + 1
        await db.execute(
            "UPDATE user_answers SET is_correct=?, wrong_streak=?, answered_a
            (int(is_correct), new_streak, existing[0])
        )
    else:
        wrong_streak = 0 if is_correct else 1
        await db.execute(
            "INSERT INTO user_answers(user_id,question_id,is_correct,wrong_st
            (user_id, question_id, int(is_correct), wrong_streak)
        )
    await db.commit()

async def db_get_leaderboard(topic_id: Optional[int] = None, limit: int = 10) ->
    async with aiosqlite.connect(DB_PATH) as db:
        if topic_id:
            async with db.execute("""
                SELECT u.id, u.username, u.first_name, COUNT(ua.id) as cnt
                FROM user_answers ua
                JOIN users u ON ua.user_id=u.id
                WHERE ua.is_correct=1 AND ua.question_id IN (
                    SELECT id FROM questions WHERE topic_id=?
                )
                GROUP BY u.id ORDER BY cnt DESC LIMIT ?
            """, (topic_id, limit)) as cur:
                rows = await cur.fetchall()
                cols = [d[0] for d in cur.description]
        else:
            async with db.execute("""
                SELECT u.id, u.username, u.first_name, u.xp as cnt
                FROM users u ORDER BY u.xp DESC LIMIT ?
            """, (limit,)) as cur:
                rows = await cur.fetchall()
                cols = [d[0] for d in cur.description]
    return [dict(zip(cols, r)) for r in rows]

async def db_get_user_rank(user_id: int) -> tuple[int, int]:
    """Возвращает (место, всего пользователей)."""
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT COUNT(*) FROM users") as cur:
            total = (await cur.fetchone())[0]
        async with db.execute(

```

```

        "SELECT COUNT(*) FROM users WHERE xp > (SELECT xp FROM users WHERE id
        (user_id,)
    ) as cur:
        above = (await cur.fetchone())[0]
    return above + 1, total

async def db_get_weak_topics(user_id: int, limit: int = 3) -> list[str]:
    """Темы с наибольшим % ошибок."""
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("""
            SELECT t.name,
                   SUM(CASE WHEN ua.is_correct=0 THEN 1 ELSE 0 END)*1.0 / COUNT(u
            FROM user_answers ua
            JOIN questions q ON ua.question_id=q.id
            JOIN topics t ON q.topic_id=t.id
            WHERE ua.user_id=?
            GROUP BY t.id
            HAVING COUNT(ua.id) >= 3
            ORDER BY err_rate DESC
            LIMIT ?
            """, (user_id, limit)) as cur:
            rows = await cur.fetchall()
    return [r[0] for r in rows]

async def db_get_strong_topics(user_id: int, limit: int = 3) -> list[str]:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("""
            SELECT t.name,
                   SUM(CASE WHEN ua.is_correct=1 THEN 1 ELSE 0 END)*1.0 / COUNT(u
            FROM user_answers ua
            JOIN questions q ON ua.question_id=q.id
            JOIN topics t ON q.topic_id=t.id
            WHERE ua.user_id=?
            GROUP BY t.id
            HAVING COUNT(ua.id) >= 3
            ORDER BY ok_rate DESC
            LIMIT ?
            """, (user_id, limit)) as cur:
            rows = await cur.fetchall()
    return [r[0] for r in rows]

async def db_get_today_count(user_id: int) -> int:
    """Сколько вопросов пользователь ответил сегодня."""
    today = datetime.utcnow().strftime("%Y-%m-%d")

```

```

async with aiosqlite.connect(DB_PATH) as db:
    async with db.execute("""
        SELECT COUNT(*) FROM user_answers
        WHERE user_id=? AND date(answered_at)=?
        """, (user_id, today)) as cur:
        row = await cur.fetchone()
    return row[0] if row else 0

async def db_get_question_by_id(question_id: int) -> Optional[dict]:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT * FROM questions WHERE id=?", (question_id,
            row = await cur.fetchone()
            if not row:
                return None
            cols = [d[0] for d in cur.description]
    return dict(zip(cols, row))

async def db_get_question_by_qid(qid: str) -> Optional[dict]:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT * FROM questions WHERE qid=?", (qid,)) as c
            row = await cur.fetchone()
            if not row:
                return None
            cols = [d[0] for d in cur.description]
    return dict(zip(cols, row))

async def db_save_question(topic_id: int, text: str, opts: list[str],
                           correct_idx: int, difficulty: str = "medium",
                           explanation: str = None) -> int:
    """Сохраняет вопрос, возвращает его id."""
    qid = "Q-" + "".join(random.choices(string.ascii_uppercase + string.digits, k
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("""
            INSERT INTO questions(qid, topic_id, question_text,
                option_a, option_b, option_c, option_d,
                correct_index, difficulty, explanation)
            VALUES(?,?,?,?,?,?,?,?,?,?)
            """, (qid, topic_id, text, opts[0], opts[1], opts[2], opts[3],
                correct_idx, difficulty, explanation)) as cur:
            qid_row = cur.lastrowid
        await db.commit()
    return qid_row

```

```

async def db_last_premium_coin_purchase(user_id: int) -> Optional[datetime]:
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("""
            SELECT purchased_at FROM premium_coin_purchases
            WHERE user_id=? ORDER BY purchased_at DESC LIMIT 1
            """, (user_id,)) as cur:
            row = await cur.fetchone()
    if not row:
        return None
    try:
        return datetime.fromisoformat(row[0])
    except Exception:
        return None

```

```

async def db_record_premium_coin_purchase(user_id: int, days: int, coins: int):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("""
            INSERT INTO premium_coin_purchases(user_id, days, coins_spent)
            VALUES(?,?,?)
            """, (user_id, days, coins))
        await db.commit()

```

```

# ┌────────────────────────────────────────────────────────────────────────────────┐
# │                                     STATES                                     │
# └────────────────────────────────────────────────────────────────────────────────┘

```

```

# ConversationHandler состояния
(
    ST_MAIN,
    ST_REASON_SELECTED,
    ST_WAITING_USER_MSG,
    ST_TOPIC_SELECT,
    ST_IN QUIZ,
    ST_APPEAL_COMMENT,
    ST_ADMIN_PANEL,
    ST_ADMIN_IMPORT_TEXT,
    ST_ADMIN_IMPORT_FILE,
    ST_ADMIN_BROADCAST,
    ST_ADMIN_FIND_USER,
    ST_ADMIN_GIVE_COINS,
    ST_ADMIN_PREMIUM,
    ST_ADMIN_REPLY,
) = range(14)

```



```

return f"{h:02d}:{m:02d}:{s:02d}"

def detect_message_type(message) -> str:
    if message.text:      return "текст"
    if message.voice:     return "голосовое"
    if message.photo:     return "фото"
    if message.video:     return "видео"
    if message.document:  return "документ"
    if message.audio:     return "аудио"
    if message.sticker:   return "стикер"
    if message.video_note: return "видеосообщение"
    if message.animation: return "GIF"
    return "другое"

async def check_channel_subscription(bot, user_id: int) -> bool:
    """Проверяет подписку на канал. Если канал не настроен – пускает всех."""
    channel = await db_get_setting("channel_id")
    if not channel:
        return True
    try:
        member = await bot.get_chat_member(chat_id=channel, user_id=user_id)
        return member.status not in (
            ChatMemberStatus.LEFT, ChatMemberStatus.BANNED
        )
    except Exception:
        return True

async def safe_send(bot, chat_id: int, text: str, **kwargs) -> bool:
    """Отправляет сообщение без краша при Forbidden."""
    try:
        await bot.send_message(chat_id=chat_id, text=text, **kwargs)
        return True
    except Forbidden:
        logger.info(f"Пользователь {chat_id} заблокировал бота")
        return False
    except Exception as e:
        logger.warning(f"safe_send {chat_id}: {e}")
        return False

async def delete_message_later(context, chat_id: int, message_id: int, delay: int)
    try:
        await asyncio.sleep(delay)
        await context.bot.delete_message(chat_id=chat_id, message_id=message_id)

```

```
except Exception:
    pass

# ════════════════════════════════════════════════════════════════════════════
# ||                                ANTI_SPAM                                ||
# ════════════════════════════════════════════════════════════════════════════

def check_rate_limit(user_id: int, max_msgs: int = 10, window: int = 10) -> bool:
    """True = ok, False = spam."""
    now = time.time()
    history = RATE_LIMIT.get(user_id, [])
    history = [t for t in history if now - t < window]
    history.append(now)
    RATE_LIMIT[user_id] = history
    return len(history) <= max_msgs

# ════════════════════════════════════════════════════════════════════════════
# ||                                KEYBOARDS (глобальные)                                ||
# ════════════════════════════════════════════════════════════════════════════

def reason_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("🔒 A) Вопрос по блокировке аккаунта", callback_data="reason:A"),
        [InlineKeyboardButton("🤝 B) Вопрос по сотрудничеству", callback_data="reason:B"),
        [InlineKeyboardButton("🧠 C) Покупка тестов", callback_data="reason:C"),
        [InlineKeyboardButton("📖 D) Свой вопрос", callback_data="reason:D"),
        [InlineKeyboardButton("📝 E) Проходить тесты ✨", callback_data="reason:E")],
    ])

def back_keyboard(cb: str = "main:menu") -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup([[InlineKeyboardButton("⬅️ Назад", callback_data=cb)])

def quiz_stop_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("⏸️ СТОП", callback_data="quiz:stop"),
         InlineKeyboardButton("🚩 Апелляция", callback_data="quiz:appeal")],
    ])

def limit_reached_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("💎 Купить Premium", callback_data="premium:shop"),
         [InlineKeyboardButton("👥 Пригласить друга", callback_data="ref:link")],
    ])
```



```
#
def get_reason_title(reason_code: str) -> str:
    return {
        "block": "🔒 Вопрос по блокировке аккаунта",
        "coop": "🤝 Вопрос по сотрудничеству",
        "tests": "🧠 Покупка тестов",
        "other": "📖 Свой вопрос",
    }.get(reason_code, "Не выбрано")

def get_reason_text(reason_code: str) -> str:
    if reason_code == "block":
        return "Напишите жалобу"
    if reason_code == "coop":
        return (
            "Хорошо 😊\n"
            "Для размещения рекламы ответьте на несколько вопросов:\n\n"
            "1 📌 Тематика рекламы.\n"
            "2 📌 Готовый рекламный пост (обязательно).\n"
            "3 📌 Срок размещения?\n"
            "4 📌 Дополнительные условия?\n\n"
            "✉ После получения информации рассчитаем стоимость."
        )
    if reason_code == "tests":
        return "Платные тесты пока недоступны 😞"
    return "Напишите ваш вопрос, и мы обязательно ответим."

def user_response_keyboard(response_id: int, selected: Optional[str] = None) -> I
left = "✅👍" if selected == "👍" else "👍"
right = "✅👏" if selected == "👏" else "👏"
return InlineKeyboardMarkup([
    InlineKeyboardButton(left, callback_data=f"userreact:{response_id}:👍"),
    InlineKeyboardButton(right, callback_data=f"userreact:{response_id}:👏"),
])

def admin_card_keyboard(request_id: int) -> InlineKeyboardMarkup:
    req = REQUESTS.get(request_id, {})
    like_count = len(req.get("admin_reactions", {}).get("👍", set()))
    heart_count = len(req.get("admin_reactions", {}).get("👏", set()))
    user_id = req.get("user", {}).get("id", 0)
    is_blocked = user_id in BLOCKED_USERS

    row2 = [InlineKeyboardButton("✅ Завершить", callback_data=f"finish:{request_
if is blocked:
```

```

        row2.append(InlineKeyboardButton("🔓 Разблокировать", callback_data=f"unk
else:
        row2.append(InlineKeyboardButton("🔒 Заблокировать", callback_data=f"blc

return InlineKeyboardMarkup([
    [InlineKeyboardButton("👉 Ответить", callback_data=f"reply:{request_id}")
    row2,
    [
        InlineKeyboardButton(f"👍 {like_count}", callback_data=f"adminreact:
        InlineKeyboardButton(f"💗 {heart_count}", callback_data=f"adminreact:
    ],
])

def build_admin_card_text(request_id: int) -> str:
    req = REQUESTS[request_id]
    user = req["user"]
    full_name = f"{escape(user.get('first_name') or '')} {escape(user.get('last_n
    uname = f"@{escape(user['username'])}" if user.get("username") else user_ment
    meta = (
        "<blockquote>"
        f"👤 Имя: {full_name}\n"
        f"💎 Username: {uname}\n"
        f"🆔 ID: <code>{user['id']}</code>\n"
        f"🔪 Причина: {escape(req['reason_title'])}\n"
        f"📌 Статус: {escape(req['status_text'])}\n"
        f"✉ Тип: {escape(req.get('message_type', 'сообщение'))}"
        "</blockquote>"
    )
    parts = ["✉ <b>Новое обращение</b>", "", meta]
    if req.get("message_text"):
        parts += ["", "<b>💬 Сообщение:</b>", escape(req["message_text"])]
    if req.get("caption"):
        parts += ["", "<b>📄 Подпись:</b>", escape(req["caption"])]
    if req.get("voice_duration"):
        parts += ["", f"<b>🎤 Голосовое:</b> {req['voice_duration']} сек."]
    admin_reacts = req.get("admin_reactions", {})
    react_parts = []
    if admin_reacts.get("👍"): react_parts.append(f"👍 {len(admin_reacts['👍'])}")
    if admin_reacts.get("💗"): react_parts.append(f"💗 {len(admin_reacts['💗'])}")
    if react_parts:
        parts += ["", "<b>🔗 Реакции:</b> " + " | ".join(react_parts)]
    return "\n".join(parts)

async def refresh_admin_cards(context, request_id: int):
    if request_id not in REQUESTS:

```

```

        return

    text = build_admin_card_text(request_id)
    markup = admin_card_keyboard(request_id)
    for item in REQUESTS[request_id].get("admin_message_refs", []):
        try:
            await context.bot.edit_message_text(
                chat_id=item["chat_id"], message_id=item["message_id"],
                text=text, parse_mode=ParseMode.HTML,
                reply_markup=markup, disable_web_page_preview=True,
            )
        except Exception as e:
            logger.warning(f"refresh admin cards: {e}")

```

```

async def send_next_question(context: ContextTypes.DEFAULT_TYPE, user_id: int, chat_id: int):
    """
    Основная функция – отправляет следующий вопрос как Quiz Poll.
    Проверяет лимиты, паузу, статус сессии.
    """
    session = await db_get_session(user_id)
    if not session or session["status"] not in ("active",):
        return

    is_prem = await db_is_premium(user_id)

    # Проверка дневного лимита
    daily_limit = int(await db_get_setting(
        "daily_limit_premium" if is_prem else "daily_limit_free",
        str(DEFAULT_DAILY_LIMIT_FREE)
    ))
    today_count = await db_get_today_count(user_id)

    if not is_prem and today_count >= daily_limit:
        # Лимит исчерпан
        await _send_limit_reached(context.bot, user_id, chat_id, session)
        await db_upsert_session(user_id, status="paused", paused_at=datetime.utcnow())
        return

    topic_id = session["topic_id"]
    q = await db_next_question(user_id, topic_id, session)
    if not q:
        # Вопросы кончились
        await send_quiz_results(context.bot, user_id, chat_id, session, reason="")

```

```

        await db_upsert_session(user_id, status="finished")
        return

options = [q["option_a"], q["option_b"], q["option_c"], q["option_d"]]

try:
    poll_msg = await context.bot.send_poll(
        chat_id=chat_id,
        question=f" ? {q['question_text']}",
        options=options,
        type="quiz",
        correct_option_id=q["correct_index"],
        is_anonymous=False,
        open_period=60,    # таймер виден прямо на Poll
        reply_markup=quiz_stop_keyboard(),
    )
    await db_upsert_session(
        user_id,
        question_id=q["id"],
        poll_message_id=poll_msg.message_id,
        chat_id=chat_id,
        missed_polls=0,    # сброс при новом вопросе
        reminder_sent=0,
    )

    # Авто-пауза через 75 сек если не ответил (65 = таймер + 5 буфер)
    context.job_queue.run_once(
        _auto_pause_check,
        when=75,
        data={"user_id": user_id, "chat_id": chat_id,
              "poll_message_id": poll_msg.message_id, "question_id": q["id"]}
        name=f"autopause_{user_id}",
    )
except Exception as e:
    logger.error(f"send_next_question error for {user_id}: {e}")

async def _auto_pause_check(context: ContextTypes.DEFAULT_TYPE):
    """Job: проверяет, ответил ли пользователь. Если нет — увеличивает missed_pol
    data = context.job.data
    user_id    = data["user_id"]
    chat_id    = data["chat_id"]
    poll_msg_id= data["poll_message_id"]
    question_id= data["question_id"]

    session = await db_get_session(user_id)
    if not session or session["status"] != "active":

```

```

        return
    # Если question_id в сессии изменился — значит пользователь уже ответил
    if session["poll_message_id"] != poll_msg_id:
        return

    # Пользователь не ответил
    missed = session.get("missed_polls", 0) + 1
    await db_upsert_session(user_id, missed_polls=missed)

    if missed >= 2:
        # Ставим на паузу
        await db_upsert_session(
            user_id, status="paused",
            paused_at=datetime.utcnow().isoformat(),
        )
        PAUSED_SESSIONS.add(user_id)
        await _send_pause_message(context.bot, user_id, chat_id, session)
        # Планируем reminder через 30 минут
        delay_min = int(await db_get_setting("reminder_delay_minutes", "30"))
        context.job_queue.run_once(
            _send_inactivity_reminder,
            when=delay_min * 60,
            data={"user_id": user_id, "chat_id": chat_id},
            name=f"reminder_{user_id}",
        )
    else:
        # Пропустил 1 — шлём следующий вопрос
        await db_upsert_session(user_id, missed_polls=missed)
        await send_next_question(context, user_id, chat_id)

async def _send_inactivity_reminder(context: ContextTypes.DEFAULT_TYPE):
    """Job: напоминание через 30 мин после паузы."""
    data = context.job.data
    user_id = data["user_id"]
    chat_id = data["chat_id"]

    session = await db_get_session(user_id)
    if not session or session["status"] != "paused":
        return
    if session.get("reminder_sent"):
        return

    await db_upsert_session(user_id, reminder_sent=1)

    rank, total = await db_get_user_rank(user_id)
    pct = max(0, 100 - round(rank / max(total, 1) * 100))

```

```

text = (
    "🔥 <b>Вы остановились почти на середине теста!</b>\n\n"
    f"✅ Правильных: {session['correct_count']}\n"
    f"🔥 Серия: {session.get('correct_count', 0)}\n"
    f"🏆 Вы обошли {pct}% пользователей\n\n"
    "Продолжите — до нового уровня осталось совсем немного!"
)

await safe_send(
    context.bot, chat_id, text,
    parse_mode=ParseMode.HTML,
    reply_markup=pause_keyboard(),
)

async def _send_pause_message(bot, user_id: int, chat_id: int, session: dict):
    """Отправляет сообщение о паузе с сохранённым прогрессом."""
    topic_name = "неизвестная"
    if session.get("topic_id"):
        async with aisqlite.connect(DB_PATH) as db:
            async with db.execute(
                "SELECT name FROM topics WHERE id=?", (session["topic_id"],)
            ) as cur:
                row = await cur.fetchone()
                if row:
                    topic_name = row[0]

    rank, total = await db_get_user_rank(user_id)

    text = (
        "⏸ <b>Тестирование приостановлено из-за неактивности</b>\n\n"
        "Ваш прогресс сохранён:\n"
        f"📖 Тема: {escape(topic_name)}\n"
        f"✅ Правильных ответов: {session['correct_count']}\n"
        f"❌ Ошибок: {session['wrong_count']}\n"
        f"🔥 Серия: {session['correct_count']}\n"
        f"🏆 Место: #{rank}"
    )

    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML, reply_markup=p

async def _send_limit_reached(bot, user_id: int, chat_id: int, session: dict):
    """Сообщение об исчерпании лимита."""
    daily_count = await db_get_today_count(user_id)
    rank, total = await db_get_user_rank(user_id)
    pct = max(0, 100 - round(rank / max(total, 1) * 100))

```

```

weak = await db_get_weak_topics(user_id, 2)
strong = await db_get_strong_topics(user_id, 1)

# Время до следующего дня (UTC)
now = datetime.utcnow()
nxt = (now + timedelta(days=1)).replace(hour=0, minute=0, second=0, microsec
diff = nxt - now
countdown = format_timedelta(diff)

weak_lines = "\n".join(f"- {w}" for w in weak) or "нет данных"
strong_line = strong[0] if strong else "нет данных"

text = (
    f"🔥 <b>Сегодня вы решили {daily_count} вопросов!</b>\n"
    f"🏆 Вы вошли в ТОП {100 - pct}%\n"
    f"📈 Серия: {session['correct_count']}\n"
    f"🧠 Сильная тема: {escape(strong_line)}\n\n"
    f"⚠️ Слабые темы:\n{escape(weak_lines)}\n\n"
    f"⌚ Лимит обновится через: <b>{countdown}</b>"
)
await safe_send(
    bot, chat_id, text,
    parse_mode=ParseMode.HTML,
    reply_markup=limit_reached_keyboard(),
)

async def _send_quiz_results(bot, user_id: int, chat_id: int, session: dict, reas
    """Полные результаты теста после СТОП или конца вопросов."""
    correct = session["correct_count"]
    wrong = session["wrong_count"]
    total = correct + wrong
    pct = round(correct / max(total, 1) * 100)

    rank, total_users = await db_get_user_rank(user_id)
    rank_pct = max(0, 100 - round(rank / max(total_users, 1) * 100))

    weak = await db_get_weak_topics(user_id, 3)
    strong = await db_get_strong_topics(user_id, 2)

    weak_lines = "\n".join(f"❌ {w}" for w in weak) or "нет данных"
    strong_lines = "\n".join(f"✅ {s}" for s in strong) or "нет данных"

    coins = session.get("coins_earned", 0)
    xp = session.get("xp_earned", 0)

    text = (

```

```

    <b>Результаты теста</b>\n"
    "\n"
    f"✅ Правильных: {correct}\n"
    f"❌ Ошибок: {wrong}\n"
    f"📈 Точность: {pct}%\n\n"
    f"🔥 Серия: {correct}\n"
    f"⭐ Получено XP: +{xp}\n"
    f"🌐 Монет заработано: +{coins}\n\n"
    f"🏆 Ваше место: #{rank} из {total_users}\n"
    f"🎯 Вы обошли {rank_pct}% участников\n\n"
    f"💪 Сильные темы:\n{strong_lines}\n\n"
    f"⚠️ Слабые темы:\n{weak_lines}"
)
await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML, reply_markup=r

```

```

# ┌────────────────────────────────────────────────────────────────────────────────┐
# │                                                                 ECONOMY │
# └────────────────────────────────────────────────────────────────────────────────┘

```

```

async def process_correct_answer(user_id: int, question: dict, session: dict) ->
    """
    Начисляет монеты и XP за правильный ответ.
    Учитывает антифарм для Premium.
    Возвращает {coins, xp, level_up}.
    """
    is_prem = await db_is_premium(user_id)
    diff = question.get("difficulty", "medium")
    rewards = DIFFICULTY_REWARDS.get(diff, DIFFICULTY_REWARDS["medium"])
    base_coins = rewards["coins"]
    base_xp = rewards["xp"]

    # x2 event
    x2 = await db_get_setting("x2_coins_active", "0") == "1"
    if x2:
        base_coins *= 2

    # Streak bonus
    streak_bonus = 0
    for streak_threshold, bonus in sorted(STREAK_BONUSES.items()):
        if session["correct_count"] > 0 and session["correct_count"] % streak_thr
            streak_bonus = bonus
            break

    # Антифарм Premium
    coin_limit = int(await db_get_setting(
        "coin_limit_premium" if is_prem else "coin_limit_free",

```

```

    str(DEFAULT_COIN_LIMIT_FREE)
))
today_count = await db_get_today_count(user_id)
coins_to_add = 0
limit_hit = False

if today_count <= coin_limit:
    coins_to_add = base_coins + streak_bonus
else:
    limit_hit = True

# Начисляем
if coins_to_add > 0:
    await db_add_coins(user_id, coins_to_add, f"quiz_correct_{question['qid']}")

new_level = await db_add_xp(user_id, base_xp)

# Обновляем streak глобальный
user = await db_get_user(user_id)
today = datetime.utcnow().strftime("%Y-%m-%d")
last = user.get("last_active", "")[:10] if user else ""
if last != today:
    yesterday = (datetime.utcnow() - timedelta(days=1)).strftime("%Y-%m-%d")
    new_streak = (user["streak"] + 1) if last == yesterday else 1
    await db_update_user(user_id, streak=new_streak, last_active=datetime.utcnow().strftime("%Y-%m-%d"))

return {
    "coins": coins_to_add,
    "xp": base_xp,
    "level_up": new_level,
    "limit_hit": limit_hit,
}

#
# || PREMIUM ||
#

async def show_premium_shop(bot, chat_id: int, user_id: int):
    """Показывает магазин Premium."""
    is_prem = await db_is_premium(user_id)
    user = await db_get_user(user_id)
    coins = user["coins"] if user else 0

    # Доступность покупки за монеты
    premium_coins_enabled = await db_get_setting("premium_coins_enabled", "1") == "1"
    last_purchase = await db_last_premium_coin_purchase(user_id)

```

```

cooldown_days = int(await db_get_setting("premium_coin_cooldown", "30"))
coin_available = (
    premium_coins_enabled
    and not is_prem
    and (
        last_purchase is None
        or (datetime.utcnow() - last_purchase).days >= cooldown_days
    )
)

coin_3d = int(await db_get_setting("premium_coin_3d", "800"))
coin_7d = int(await db_get_setting("premium_coin_7d", "1800"))

status_line = "✅ У вас активен Premium!" if is_prem else "❌ У вас нет Premi

text = (
    "💎 <b>Premium-подписка</b>\n\n"
    f"{status_line}\n"
    f"🌐 Ваш баланс: {coins} монет\n\n"
    "🚀 <b>Преимущества Premium:</b>\n"
    "    ✅ Безлимитные тесты\n"
    "    ✅ Закрытые темы\n"
    "    ✅ Расширенная статистика\n"
    "    ✅ Premium-рейтинг\n"
    "    ✅ Приоритетная поддержка\n\n"
    "💰 <b>Тарифы (за реальные деньги):</b>\n"
    "    🟢 3 дня — 490 тенге\n"
    "    🔵 7 дней — 890 тенге\n"
    "    🟣 30 дней — 1990 тенге ⭐\n"
    "    🏆 90 дней — 3990 тенге 🏆\n"
)

if coin_available:
    text += (
        f"\n🌐 <b>Купить за монеты (раз в {cooldown_days} дн.):</b>\n"
        f"    • 3 дня — {coin_3d} монет\n"
        f"    • 7 дней — {coin_7d} монет\n"
        "<i>⚠️ Только как пробный доступ. Для постоянного Premium покупайте э
    )

buttons = []
if not is_prem:
    buttons.append([
        InlineKeyboardButton("🟢 3 дня — 490 тг", callback_data="buy_prem:3d"),
        InlineKeyboardButton("🟣 30 дней — 1990 тг", callback_data="buy_prem:30d"),
    ])
    buttons.append([

```

```

        InlineKeyboardButton("🇷🇺 7 дней – 890 тг", callback_data="buy_prem:r
        InlineKeyboardButton("🇷🇺 90 дней – 3990 тг", callback_data="buy_prem:
    ])
    if coin_available:
        buttons.append([
            InlineKeyboardButton(f"🌐 3 дня за {coin_3d} монет", callback_da
            InlineKeyboardButton(f"🌐 7 дней за {coin_7d} монет", callback_da
        ])

    buttons.append([InlineKeyboardButton("⬅️ Назад", callback_data="main:menu")])

    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=InlineKeyboardMarkup(buttons))

async def handle_buy_premium_coins(bot, user_id: int, chat_id: int, tier: str):
    """Покупка Premium за монеты."""
    if tier not in PREMIUM_COIN_TIERS:
        await safe_send(bot, chat_id, "❌ Неверный тариф.")
        return

    tier_info = PREMIUM_COIN_TIERS[tier]
    days = tier_info["days"]
    cost = int(await db_get_setting(f"premium_coin_{tier}", str(tier_info["coins

    # Проверка cooldown
    last_purchase = await db_last_premium_coin_purchase(user_id)
    cooldown_days = int(await db_get_setting("premium_coin_cooldown", "30"))
    if last_purchase and (datetime.utcnow() - last_purchase).days < cooldown_days
        days_left = cooldown_days - (datetime.utcnow() - last_purchase).days
        await safe_send(bot, chat_id,
            f"❌ Купить Premium за монеты можно раз в {cooldown_days} дней.\n"
            f"Осталось ждать: {days_left} дн."
        )
    return

    # Проверка активного Premium
    if await db_is_premium(user_id):
        await safe_send(bot, chat_id, "❌ У вас уже активен Premium.")
        return

    # Проверка баланса
    user = await db_get_user(user_id)
    if not user or user["coins"] < cost:
        await safe_send(bot, chat_id,
            f"❌ Недостаточно монет.\n"
            f"Нужно: {cost}\nВаш баланс: {user['coins'] if user else 0}"

```

```

    )
    return

# Списываем монеты, выдаём Premium
await db_add_coins(user_id, -cost, f"buy_premium_{tier}")
await db_grant_premium(user_id, days, source="coins")
await db_record_premium_coin_purchase(user_id, days, cost)

expires = datetime.utcnow() + timedelta(days=days)
text = (
    f"🎉 <b>Premium на {days} дней активирован!</b>\n\n"
    f"🌐 Списано: {cost} монет\n"
    f"📅 Действует до: {expires.strftime('%d.%m.%Y %H:%M')} UTC\n\n"
    "<i>Следующая покупка за монеты будет доступна через 30 дней.\n"
    "Для постоянного Premium рекомендуем покупку за деньги!</i>"
)
await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                reply_markup=back_keyboard("main:menu"))

# Уведомление админу
for adm_id in ADMINS:
    await safe_send(
        bot, adm_id,
        f"💜 <b>Новый Premium за монеты</b>\n\n"
        f"ID: <code>{user_id}</code>\n"
        f"Username: @{{escape(user.get('username') or 'нет')}}\n"
        f"Тариф: {tier_info['label']}\n"
        f"Списано: {cost} монет",
        parse_mode=ParseMode.HTML,
    )

async def handle_buy_premium_money(bot, user_id: int, chat_id: int, days: int):
    """Запрос на покупку Premium за деньги — уведомляет админа."""
    user = await db_get_user(user_id)
    uname = f"@{user['username']}" if user and user.get("username") else "нет"

    text = (
        f"✉️ <b>Запрос на покупку Premium</b>\n\n"
        f"ID: <code>{user_id}</code>\n"
        f"Username: {escape(uname)}\n"
        f"Имя: {escape(user.get('first_name', '')) if user else ''}\n"
        f"Тариф: {days} дней\n\n"
        "Ожидайте — администратор свяжется с вами для оплаты."
    )
    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=back_keyboard("main:menu"))

```

```

for adm_id in ADMINS:
    await safe_send(
        bot, adm_id,
        f"👤 <b>Запрос Premium за деньги</b>\n\n"
        f"ID: <code>{user_id}</code>\n"
        f"Username: {escape(uname)}\n"
        f"Имя: {escape(user.get('first_name', '')) if user else ''}\n"
        f"Тариф: {days} дней",
        parse_mode=ParseMode.HTML,
    )

```

```

async def check_expired_premiums(context: ContextTypes.DEFAULT_TYPE):
    """Job: проверяет истёкшие Premium и уведомляет пользователей."""
    now = datetime.utcnow().isoformat()
    async with aisqlite.connect(DB_PATH) as db:
        async with db.execute("""
            SELECT user_id FROM premium
            WHERE expires_at < ? AND notified_expire=0
            """, (now,)) as cur:
            rows = await cur.fetchall()

    for (user_id,) in rows:
        user = await db_get_user(user_id)
        solved = 0
        async with aisqlite.connect(DB_PATH) as db:
            async with db.execute(
                "SELECT COUNT(*) FROM user_answers WHERE user_id=? AND is_correct"
                "(user_id,)"
            ) as cur:
                row = await cur.fetchone()
                if row:
                    solved = row[0]

        rank, total = await db_get_user_rank(user_id)
        pct = max(0, 100 - round(rank / max(total, 1) * 100))

        text = (
            f"⚠️ <b>Premium закончился</b>\n\n"
            f"Вы активно пользовались Premium:\n"
            f"✅ Решено вопросов: {solved}\n"
            f"🏆 Топ {100 - pct}%\n"
            f"📊 Расширенная статистика была активна\n\n"
            f"Чтобы продолжить без ограничений, купите Premium!"
        )

    buttons = InlineKeyboardMarkup([

```

```

        [InlineKeyboardButton("🛒 Купить Premium", callback_data="pr
        [InlineKeyboardButton("👥 Пригласить друга", callback_data="re
    ])
    sent = await safe_send(
        context.bot, user_id, text,
        parse_mode=ParseMode.HTML, reply_markup=buttons
    )
    if sent:
        async with aisqlite.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE premium SET notified_expire=1 WHERE user_id=?", (user
            )
            await db.commit()

async def notify_limit_restored(context: ContextTypes.DEFAULT_TYPE):
    """Job: уведомляет пользователей о восстановлении лимита (один раз в сутки)."""
    today = datetime.utcnow().strftime("%Y-%m-%d")
    # Ищем тех, у кого вчера был active или paused сеанс, но НЕ premium
    async with aisqlite.connect(DB_PATH) as db:
        async with db.execute("""
            SELECT s.user_id, s.chat_id, s.topic_id, s.correct_count, s.status
            FROM sessions s
            LEFT JOIN premium p ON p.user_id=s.user_id
            WHERE (p.user_id IS NULL OR p.expires_at < ?)
            AND s.status='paused'
            AND date(s.session_date) < ?
            """, (today, today)) as cur:
            rows = await cur.fetchall()
            cols = [d[0] for d in cur.description]

    for row in rows:
        sess = dict(zip(cols, row))
        user_id = sess["user_id"]
        chat_id = sess.get("chat_id") or user_id
        topic_name = ""
        if sess.get("topic_id"):
            async with aisqlite.connect(DB_PATH) as db:
                async with db.execute(
                    "SELECT name FROM topics WHERE id=?", (sess["topic_id"],)
                ) as cur:
                    r = await cur.fetchone()
                    if r:
                        topic_name = r[0]

    text = (
        🎉 <b>Ваш лимит вопросов восстановлен!</b>\n\n"

```

```

        "Теперь вы снова можете продолжить тестирование.\n\n"
        "Ваш прогресс сохранён:\n"
        f" Тема: {escape(topic_name)}\n"
        f" Правильных ответов: {sess['correct_count']}"
    )
    sent = await safe_send(
        context.bot, chat_id, text,
        parse_mode=ParseMode.HTML,
        reply_markup=pause_keyboard(),
    )
    if sent:
        await db_upsert_session(user_id, session_date=today)

#
# || REFERRALS ||
#

async def process_referral(user_id: int, ref_code: str, bot):
    """Обрабатывает реферальную ссылку при /start ref_XXXX."""
    # Ищем реферера
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT id FROM users WHERE ref_code=?", (ref_code,)
        ) as cur:
            row = await cur.fetchone()

    if not row:
        return
    referrer_id = row[0]

    # Нельзя пригласить самого себя
    if referrer_id == user_id:
        return

    # Уже был реферал?
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT id FROM referrals WHERE referred_id=?", (user_id,)
        ) as cur:
            existing = await cur.fetchone()

    if existing:
        return

    # Проверяем дневной лимит рефералов
    today = datetime.utcnow().strftime("%Y-%m-%d")

```

```

async with aioredis.connect(DB_PATH) as db:
    async with db.execute("""
        SELECT COUNT(*) FROM referrals
        WHERE referrer_id=? AND date(created_at)=?
        """, (referrer_id, today)) as cur:
        today_refs = (await cur.fetchone())[0]

daily_ref_limit = int(await db_get_setting("referral_limit_per_day", "5"))
if today_refs >= daily_ref_limit:
    return

# Начисляем монеты
coin_per_ref = int(await db_get_setting("coin_per_referral", "200"))
async with aioredis.connect(DB_PATH) as db:
    await db.execute(
        "INSERT INTO referrals(referrer_id, referred_id, coins_given) VALUES(
            (referrer_id, user_id, coin_per_ref)
        )
    )
    await db.commit()

await db_add_coins(referrer_id, coin_per_ref, f"referral_{user_id}")
await db_update_user(user_id, invited_by=referrer_id)

# Уведомляем реферера
referred_user = await db_get_user(user_id)
ref_name = escape(referred_user.get("first_name", "Новый участник") if referr
await safe_send(
    bot, referrer_id,
    f"По вашей ссылке зарегистрировался новый участник!\n\n",
    f"{ref_name}\n",
    f"Вам начислено: +{coin_per_ref} монет",
    parse_mode=ParseMode.HTML,
)

```

```

def parse_questions_text(raw: str) -> list[dict]:
    """
    Парсит текст формата:
    1. Вопрос?
    A) Ответ 1*
    B) Ответ 2
    C) Ответ 3
    D) Ответ 4
    """
    
```

```

* = правильный ответ
"""
questions = []
# Разбиваем по номерам вопросов
blocks = re.split(r'\n(?:\d+[\.\.])\s)', raw.strip())

for block in blocks:
    lines = [l.strip() for l in block.strip().splitlines() if l.strip()]
    if not lines:
        continue

    # Первая строка — вопрос (убираем номер)
    q_line = re.sub(r'^\d+[\.\.]\s*', '', lines[0])
    if not q_line:
        continue

    opts = []
    correct_idx = 0
    for line in lines[1:]:
        # Убираем метку варианта (A), B), C), D), 1), A), и т.п.)
        m = re.match(r'^[ABCDaБбВвГгDd1-4][\.\.]?s*(.*)', line, re.IGNORECASE)
        if not m:
            continue
        opt_text = m.group(1).strip()
        is_correct = opt_text.endswith("*")
        if is_correct:
            opt_text = opt_text[:-1].strip()
            correct_idx = len(opts)
        opts.append(opt_text)

    if len(opts) == 4 and q_line:
        questions.append({
            "text": q_line,
            "opts": opts,
            "correct": correct_idx,
        })

return questions

```

```

async def save_draft(admin_id: int, questions: list[dict], topic_id: int) -> int:
    data = json.dumps({"questions": questions, "topic_id": topic_id}, ensure_ascii=False)
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "INSERT INTO drafts(admin_id, data) VALUES(?,?)", (admin_id, data)
        ) as cur:

```

```

        draft_id = cur.lastrowid
    await db.commit()
return draft_id


async def get_draft(draft_id: int) -> Optional[dict]:
    async with aioredis.connect(DB_PATH) as db:
        async with db.execute("SELECT data FROM drafts WHERE id=?", (draft_id,))
            row = await cur.fetchone()
    if not row:
        return None
    return json.loads(row[0])


async def commit_draft(draft_id: int):
    """Сохраняет черновик в основную таблицу вопросов."""
    draft = await get_draft(draft_id)
    if not draft:
        return 0
    topic_id = draft["topic_id"]
    questions = draft["questions"]
    saved = 0
    for q in questions:
        try:
            await db_save_question(topic_id, q["text"], q["opts"], q["correct"])
            saved += 1
        except Exception as e:
            logger.warning(f"commit_draft question error: {e}")

    async with aioredis.connect(DB_PATH) as db:
        await db.execute("DELETE FROM drafts WHERE id=?", (draft_id,))
        await db.commit()
    return saved


# ┌────────────────────────────────────────────────────────────────────────────────┐
# │                                                                 ADMIN PANEL │
# └────────────────────────────────────────────────────────────────────────────────┘


async def show_admin_panel(bot, chat_id: int):
    await safe_send(
        bot, chat_id,
        "👤 <b>Панель администратора</b>\n\nВыберите раздел:",
        parse_mode=ParseMode.HTML,
        reply_markup=admin_main_keyboard(),
    )
```

```

async def adm_show_stats(bot, chat_id: int):
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT COUNT(*) FROM users") as cur:
            total_users = (await cur.fetchone())[0]
        async with db.execute(
            "SELECT COUNT(*) FROM premium WHERE expires_at > ?",
            (datetime.utcnow().isoformat(),)
        ) as cur:
            premium_count = (await cur.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM user_answers") as cur:
            total_answers = (await cur.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM questions WHERE is_active=1")
            total_questions = (await cur.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM topics WHERE is_active=1") as
            total_topics = (await cur.fetchone())[0]
        async with db.execute(
            "SELECT COUNT(*) FROM users WHERE last_active > ?",
            ((datetime.utcnow() - timedelta(days=1)).isoformat(),)
        ) as cur:
            dau = (await cur.fetchone())[0]
        async with db.execute(
            "SELECT COUNT(*) FROM users WHERE last_active > ?",
            ((datetime.utcnow() - timedelta(days=30)).isoformat(),)
        ) as cur:
            mau = (await cur.fetchone())[0]
        async with db.execute("SELECT SUM(coins) FROM users") as cur:
            total_coins = (await cur.fetchone())[0] or 0

    text = (
        f"<b>Статистика бота</b>\n\n"
        f"👥 Всего пользователей: {total_users}\n"
        f"💎 Premium-пользователей: {premium_count}\n"
        f"📅 DAU (сегодня): {dau}\n"
        f"📅 MAU (месяц): {mau}\n\n"
        f"🧠 Вопросов в базе: {total_questions}\n"
        f"📖 Тем: {total_topics}\n"
        f"✅ Всего ответов: {total_answers}\n\n"
        f"🌐 Монет в обороте: {total_coins:}"
    )

    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=back_keyboard("adm:main"))

async def adm_show_tests(bot, chat_id: int):
    topics = await db_get_topics(premium_ok=True)
    if not topics:

```

```

        await safe_send(bot, chat_id, "🚫 Нет тем. Создайте тему через импорт.",
                        reply_markup=back_keyboard("adm:main"))

    return

lines = []
for t in topics:
    cnt = await db_count_questions(t["id"])
    prem = "💎" if t["is_premium"] else "🆓"
    lines.append(f"{prem} {escape(t['name'])} — {cnt} вопросов (ID:{t['id']})")

buttons = [
    [InlineKeyboardButton("✚ Создать тему", callback_data="adm:tests:new_top"),
     [InlineKeyboardButton("🔍 Найти вопрос по ID", callback_data="adm:tests:f"),
      [InlineKeyboardButton("⬅ Назад", callback_data="adm:main")]],
    ]
text = "🧠 <b>Темы и вопросы</b>\n\n" + "\n".join(lines)
await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                reply_markup=InlineKeyboardMarkup(buttons))

async def adm_show_import_menu(bot, chat_id: int):
    text = (
        "📁 <b>Импорт вопросов</b>\n\n"
        "Выберите способ импорта:"
    )
    buttons = [
        [InlineKeyboardButton("📄 Импорт текстом", callback_data="adm:import:t"),
         [InlineKeyboardButton("📊 Импорт Quiz Poll", callback_data="adm:import:p"),
          [InlineKeyboardButton("⬅ Назад", callback_data="adm:main")]],
        ]
    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=InlineKeyboardMarkup(buttons))

async def adm_show_users(bot, chat_id: int, admin_id: int):
    text = (
        "👥 <b>Управление пользователями</b>\n\n"
        "Введите ID или username для поиска:"
    )
    buttons = [
        [InlineKeyboardButton("🔍 Найти пользователя", callback_data="adm:users:f"),
         [InlineKeyboardButton("🏆 Топ по XP", callback_data="adm:users:t"),
          [InlineKeyboardButton("⬅ Назад", callback_data="adm:main")]]
        ]
    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=InlineKeyboardMarkup(buttons))

```

```

async def adm_show_appeals(bot, chat_id: int):
    async with aioredis.connect(DB_PATH) as db:
        async with db.execute("""
            SELECT a.id, a.user_id, q.qid, a.status, a.comment
            FROM appeals a
            JOIN questions q ON a.question_id=q.id
            WHERE a.status='pending'
            ORDER BY a.created_at DESC LIMIT 10
        """) as cur:
            rows = await cur.fetchall()

    if not rows:
        await safe_send(bot, chat_id, "✅ Новых апелляций нет.",
                        reply_markup=back_keyboard("adm:main"))
        return

    lines = []
    buttons = []
    for ap_id, user_id, qid, status, comment in rows:
        lines.append(f"#{ap_id} | {qid} | user:{user_id}\n💬 {escape(comment or ' ')}")
        buttons.append([
            InlineKeyboardButton(f"#{ap_id} → {qid}", callback_data=f"adm:appeal:
        ])

    buttons.append([InlineKeyboardButton("⬅️ Назад", callback_data="adm:main")])
    text = "▶️ <b>Апелляции (ожидают рассмотрения)</b>\n\n" + "\n\n".join(lines)
    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=InlineKeyboardMarkup(buttons))

async def adm_show_appeal(bot, chat_id: int, appeal_id: int):
    async with aioredis.connect(DB_PATH) as db:
        async with db.execute("""
            SELECT a.*, q.qid, q.question_text,
                   q.option_a, q.option_b, q.option_c, q.option_d,
                   q.correct_index, t.name as topic_name
            FROM appeals a
            JOIN questions q ON a.question_id=q.id
            JOIN topics t ON q.topic_id=t.id
            WHERE a.id=?
        """, (appeal_id,)) as cur:
            row = await cur.fetchone()
            if not row:
                await safe_send(bot, chat_id, "❌ Апелляция не найдена.")
                return
            cols = [d[0] for d in cur.description]

```

```

ap = dict(zip(cols, row))

opts = [ap["option_a"], ap["option_b"], ap["option_c"], ap["option_d"]]
opts_text = "\n".join(
    f" {img('✅')} if i == ap['correct_index'] else ' ' {chr(65+i)) {escape(o)}
    for i, o in enumerate(opts)
)
text = (
    f"🚩 <b>Апелляция #{appeal_id}</b>\n\n"
    f"ID вопроса: <code>{ap['qid']}</code>\n"
    f"Тема: {escape(ap['topic_name'])}\n"
    f"👤 Пользователь: <code>{ap['user_id']}</code>\n\n"
    f"? {escape(ap['question_text'])}\n\n"
    f"{opts_text}\n\n"
    f"💬 Комментарий: {escape(ap['comment'] or '-')}\n"
    f"📖 Источник: {escape(ap['source_ref'] or '-')}"
)
buttons = InlineKeyboardMarkup([
    [
        InlineKeyboardButton("✅ Принять", callback_data=f"adm:appeal:accep
        InlineKeyboardButton("❌ Отклонить", callback_data=f"adm:appeal:rejec
    ],
    [InlineKeyboardButton("✎ Изменить ответ", callback_data=f"adm:appeal:edi
    [InlineKeyboardButton("🗑 Удалить вопрос", callback_data=f"adm:appeal:de
    [InlineKeyboardButton("⬅ Назад", callback_data="adm:appeals")]
])
await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML, reply_markup=b

```

```

async def adm_show_economy(bot, chat_id: int):
    coin_free = await db_get_setting("coin_limit_free", str(DEFAULT_COIN_LIMIT
    coin_prem = await db_get_setting("coin_limit_premium", str(DEFAULT_COIN_LI
    limit_free = await db_get_setting("daily_limit_free", str(DEFAULT_DAILY_LIM
    coin_ref = await db_get_setting("coin_per_referral", "200")
    x2 = await db_get_setting("x2_coins_active", "0")
    prem_coins = await db_get_setting("premium_coins_enabled", "1")
    coin_3d = await db_get_setting("premium_coin_3d", "800")
    coin_7d = await db_get_setting("premium_coin_7d", "1800")
    cooldown = await db_get_setting("premium_coin_cooldown", "30")

    text = (
        f"🌐 <b>Настройки экономики</b>\n\n"
        f"🆓 ЛИМИТ вопросов (free): {limit_free}/день\n"
        f"🌐 ЛИМИТ монет (free): {coin_free}/день\n"
        f"💎 ЛИМИТ монет (premium): {coin_prem}/день\n"
        f"👥 Монет за реферала: {coin_ref}\n"
        f"✖ x2 монеты: {'🟢 ВКЛ' if x2=='1' else '🔴 ВЫКЛ'}\n\n"
    )

```

```
f"<img alt="blue diamond icon" data-bbox="108 0 135 15"/> Premium за монеты: {'<img alt="green circle icon" data-bbox="418 0 445 15"/> ВКЛ' if prem_coins=='1' else '<img alt="red circle icon" data-bbox="825 0 852 15"/> ВЫКЛ'}\n"
f"    • 3 дня: {coin_3d} монет\n"
f"    • 7 дней: {coin_7d} монет\n"
f"    • Cooldown: {cooldown} дней"
)
buttons = InlineKeyboardMarkup([
    [InlineKeyboardButton("<img alt='X icon'" data-bbox="108 132 135 147"/> Переключить x2 монеты", callback_data="adm:eco:
    [InlineKeyboardButton("<img alt='diamond icon'" data-bbox="108 152 135 167"/> Переключить Premium за монеты", callback_data="
    [InlineKeyboardButton("<img alt='back arrow icon'" data-bbox="108 172 135 187"/> Назад", callback_data="adm:main")]],
])
await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML, reply_markup=b

async def adm_show_settings(bot, chat_id: int):
    channel      = await db_get_setting("channel_id", "не задан")
    limit_free   = await db_get_setting("daily_limit_free", str(DEFAULT_DAILY_LIMIT
    closed       = await db_get_setting("closed_mode", "0")
    antifarm     = await db_get_setting("antifarm_enabled", "1")

    text = (
        "<img alt='gear icon'" data-bbox="108 428 135 443"/> <b>Настройки бота</b>\n\n"
        f"<img alt='megaphone icon'" data-bbox="108 448 135 463"/> Канал подписки: {channel}\n"
        f"<img alt='1234 icon'" data-bbox="108 468 135 483"/> Лимит вопросов (free): {limit_free}/день\n"
        f"<img alt='lock icon'" data-bbox="108 488 135 503"/> Закрытый режим: {'<img alt='green circle icon'" data-bbox="418 493 445 508"/> ВКЛ' if closed=='1' else '<img alt='red circle icon'" data-bbox="825 493 852 508"/> ВЫКЛ'}\n"
        f"<img alt='shield icon'" data-bbox="108 508 135 523"/> Антифарм: {'<img alt='green circle icon'" data-bbox="418 513 445 528"/> ВКЛ' if antifarm=='1' else '<img alt='red circle icon'" data-bbox="825 513 852 528"/> ВЫКЛ'}"
    )
    buttons = InlineKeyboardMarkup([
        [InlineKeyboardButton("<img alt='lock icon'" data-bbox="108 578 135 593"/> Переключить закрытый режим", callback_data="adr
        [InlineKeyboardButton("<img alt='shield icon'" data-bbox="108 598 135 613"/> Переключить антифарм",          callback_data="adr
        [InlineKeyboardButton("<img alt='back arrow icon'" data-bbox="108 618 135 633"/> Назад",                callback_data="adr
    ])
    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML, reply_markup=b

# |||
# || HANDLERS |||
# |||

async def cmd_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not update.message:
        return
    user = update.effective_user
    await db_get_or_create_user(user)

# Реферальная обработка
args = context.args or []
if args and args[0].startswith("ref "):
```

```

        ref_code = args[0][4:]
        await process_referral(user.id, ref_code, context.bot)

    if is_admin(user.id):
        await update.message.reply_text(
            "✅ <b>Вы вошли как администратор</b>\n\n"
            "Используйте /admin для панели управления.",
            parse_mode=ParseMode.HTML,
        )
        return

    # Обновляем статус
    FINISHED_USERS.discard(user.id)
    context.user_data.clear()
    context.user_data["started"] = True

    await update.message.reply_text(
        f"👋 Здравствуйте, {safe_username(user)}!\n\nВыберите действие:",
        reply_markup=reason_keyboard(),
        parse_mode=ParseMode.HTML,
    )

async def cmd_admin(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not update.message:
        return
    if not is_admin(update.effective_user.id):
        await update.message.reply_text("🚫 Команда недоступна.")
        return
    await show_admin_panel(context.bot, update.effective_chat.id)

async def cmd_id(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    await update.message.reply_text(
        f"🆔 Ваш Telegram ID: <code>{user.id}</code>",
        parse_mode=ParseMode.HTML,
    )

async def cmd_cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    if not is_admin(user.id):
        await update.message.reply_text("🚫 Команда доступна только администратор")
        return
    context.user_data.pop("reply_request_id", None)
    context.user_data.pop("admin_import_topic_id", None)

```

```

context.user_data.pop("admin_broadcast_mode", None)
await update.message.reply_text("✅ Режим отменён.")

async def cmd_profile(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not update.message:
        return
    user_id = update.effective_user.id
    user = await db_get_user(user_id)
    if not user:
        await update.message.reply_text("❌ Профиль не найден. Нажмите /start")
        return

    is_prem = await db_is_premium(user_id)
    lvl, lvl_name, xp_in_lvl, xp_to_next = get_level_info(user["xp"])
    rank, total = await db_get_user_rank(user_id)

    prem_line = "✅ Premium активен" if is_prem else "❌ нет Premium"
    text = (
        f"👤 <b>{safe_username(update.effective_user)}</b>\n"
        f"_____ \n"
        f"🏆 Уровень: {lvl_name}\n"
        f"⭐ XP: {user['xp']:,} / +{xp_to_next} до следующего\n"
        f"🔥 Серия: {user['streak']} дней\n"
        f"🌐 Монет: {user['coins']:,}\n\n"
        f"🏆 Рейтинг: #{rank} из {total}\n"
        f"💎 {prem_line}\n\n"
        f"🔗 Ваша реферальная ссылка:\n"
        f"<code>https://t.me/{(await context.bot.get_me()).username}?start=ref_{u"
    )
    buttons = InlineKeyboardMarkup([
        [InlineKeyboardButton("📊 Статистика", callback_data="profile:stats"),
          InlineKeyboardButton("🏆 Рейтинг", callback_data="profile:rating"),
          InlineKeyboardButton("💎 Premium", callback_data="premium:shop"),
          InlineKeyboardButton("👤 Рефералы", callback_data="ref:link")],
        [InlineKeyboardButton("🏠 Меню", callback_data="main:menu")],
    ])
    await update.message.reply_text(text, parse_mode=ParseMode.HTML, reply_markup

async def cmd_banlist(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not is_admin(update.effective_user.id):
        await update.message.reply_text("🚫 Только для администраторов.")
        return
    if not BLOCKED_USERS:
        await update.message.reply_text("✅ Банлист пуст.")
        return

```

```

lines = ["🚫 <b>Банлист</b>"]
for uid in BLOCKED_USERS:
    user = await db_get_user(uid)
    if user:
        name = escape(user.get("first_name") or "-")
        uname = f"@{escape(user['username'])}" if user.get("username") else ""
        lines.append(f"• {name} | {uname} | <code>{uid}</code>")
    else:
        lines.append(f"• <code>{uid}</code>")
await update.message.reply_text(
    "\n".join(lines), parse_mode=ParseMode.HTML, disable_web_page_preview=True
)

```

— POLL ANSWER HANDLER —————

```

async def handle_poll_answer(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Обрабатывает ответ пользователя на Quiz Poll."""
    poll_answer = update.poll_answer
    if not poll_answer:
        return

    user_id = poll_answer.user.id
    chosen = poll_answer.option_ids

    # Если пользователь отозвал ответ (пустой список) — игнорируем
    if not chosen:
        return

    # Отменяем scheduled autopause
    jobs = context.job_queue.get_jobs_by_name(f"autopause_{user_id}")
    for job in jobs:
        job.schedule_removal()

    session = await db_get_session(user_id)
    if not session or session["status"] != "active":
        return

    question = await db_get_question_by_id(session["question_id"])
    if not question:
        return

    is_correct = chosen[0] == question["correct_index"]
    chat_id = session["chat_id"]

    # Запись ответа
    await db_record_answer(user_id, question["id"], is_correct)

```

```

if is_correct:
    # Начисляем награды
    rewards = await process_correct_answer(user_id, question, session)
    coins = rewards["coins"]
    xp = rewards["xp"]
    level_up = rewards["level_up"]

    new_correct = session["correct_count"] + 1
    await db_upsert_session(
        user_id,
        correct_count=new_correct,
        total_asked=session["total_asked"] + 1,
        coins_earned=session["coins_earned"] + coins,
        xp_earned=session["xp_earned"] + xp,
        missed_polls=0,
    )

    # Единственное сообщение которое показываем — новый уровень
    if level_up:
        lvl_name = LEVELS[level_up][1]
        await safe_send(
            context.bot, chat_id,
            f"
```

```

        return
    await query.answer()

    data = query.data
    user = update.effective_user
    chat_id = query.message.chat_id if query.message else user.id

    # — Антиспам —————
    if not check_rate_limit(user.id, max_msgs=20, window=5):
        await query.answer("⚠ Слишком много запросов. Подождите.", show_alert=True)
        return

    # — Главное меню —————
    if data == "main:menu":
        try:
            await query.edit_message_text(
                f"👉 Выберите действие:",
                reply_markup=reason_keyboard(),
            )
        except Exception:
            await safe_send(context.bot, chat_id, "👉 Выберите действие:",
                            reply_markup=reason_keyboard())
        return

    # — Выбор причины (A/B/C/D) —————
    if data.startswith("reason:"):
        await _cb_reason(query, context, user, chat_id, data)
        return

    # — Квиз —————
    if data.startswith("quiz:"):
        await _cb_quiz(query, context, user, chat_id, data)
        return

    # — Premium —————
    if data.startswith("premium:") or data.startswith("buy_prem:"):
        await _cb_premium(query, context, user, chat_id, data)
        return

    # — Профиль —————
    if data.startswith("profile:"):
        await _cb_profile(query, context, user, chat_id, data)
        return

    # — Рефералы —————
    if data.startswith("ref:"):
        await _cb_referral(query, context, user, chat_id, data)

```

```

        return

# — Старые обращения (A/B/C/D) —————
if data.startswith("reply:"):
    await handle_reply_button(update, context)
    return
if data.startswith("adminreact:"):
    await handle_admin_reaction(update, context)
    return
if data.startswith("userreact:"):
    await handle_user_reaction(update, context)
    return
if data.startswith("block:"):
    await handle_block_user(update, context)
    return
if data.startswith("unblock:"):
    await handle_unblock_user(update, context)
    return
if data.startswith("finish:"):
    await handle_finish_dialog(update, context)
    return

# — Админка —————
if data.startswith("adm:"):
    await _cb_admin(query, context, user, chat_id, data)
    return

# — Апелляции (пользователь) —————
if data.startswith("topic:"):
    await _cb_topic_select(query, context, user, chat_id, data)
    return

if data.startswith("draft:"):
    await _cb_draft(query, context, user, chat_id, data)
    return

async def _cb_reason(query, context, user, chat_id, data):
    """Обработка выбора причины (A/B/C/D)."""
    if is_admin(user.id) or user.id in BLOCKED_USERS:
        return

    if data == "reason:back":
        context.user_data["reason_selected"] = False
        context.user_data.pop("reason_code", None)
        try:
            await query.edit_message_text("Выберите причину обращения:", reply_ma

```

```

        except Exception:
            pass
        return

reason_code = data.split(":", 1)[1]
reason_title = get_reason_title(reason_code)

context.user_data["started"] = True
context.user_data["reason_selected"] = True
context.user_data["reason_code"] = reason_code
context.user_data["reason_title"] = reason_title

try:
    await query.edit_message_text(f"✅ Причина: {reason_title}")
except Exception:
    pass

await safe_send(
    context.bot, chat_id,
    get_reason_text(reason_code),
    reply_markup=back_keyboard("reason:back"),
)

async def _cb_quiz(query, context, user, chat_id, data):
    """Обработка всех quiz: callback."""
    user_id = user.id

    if data == "quiz:check_sub":
        ok = await check_channel_subscription(context.bot, user_id)
        if ok:
            await query.answer("✅ Подписка подтверждена!", show_alert=True)
            await _start_quiz_flow(context, user_id, chat_id)
        else:
            await query.answer("❌ Вы не подписаны на канал.", show_alert=True)
        return

    if data == "quiz:start":
        if user_id in BLOCKED_USERS:
            await query.answer("🚫 Вы заблокированы.", show_alert=True)
            return
        # Проверка подписки
        ok = await check_channel_subscription(context.bot, user_id)
        if not ok:
            channel = await db_get_setting("channel_id", "")
            await safe_send(
                context.bot, chat_id,

```

```

        "🔊 <b>Для доступа к тестам необходимо подписаться на канал</b>",
        parse_mode=ParseMode.HTML,
        reply_markup=subscription_keyboard(channel),
    )
    return
await _start_quiz_flow(context, user_id, chat_id)
return

if data == "quiz:resume":
    session = await db_get_session(user_id)
    if not session:
        await query.answer("❌ Нет активной сессии.", show_alert=True)
        return
    await db_upsert_session(user_id, status="active", missed_polls=0, remind=
PAUSED_SESSIONS.discard(user_id)
    await send_next_question(context, user_id, chat_id)
    return

if data == "quiz:stop":
    session = await db_get_session(user_id)
    if not session:
        await query.answer("❌ Нет активного теста.", show_alert=True)
        return
    await db_upsert_session(user_id, status="finished")
    await _send_quiz_results(context.bot, user_id, chat_id, session, reason="
    return

if data == "quiz:appeal":
    session = await db_get_session(user_id)
    if not session or not session.get("question_id"):
        await query.answer("❌ Нет активного вопроса.", show_alert=True)
        return
    context.user_data["appeal_question_id"] = session["question_id"]
    await db_upsert_session(user_id, status="paused")
    await safe_send(
        context.bot, chat_id,
        "🔊 <b>Апелляция</b>\n\n"
        "Тест приостановлен.\n"
        "Опишите ошибку и укажите источник.\n"
        "Формат:\n<code>Комментарий | Источник (книга, страница)</code>",
        parse_mode=ParseMode.HTML,
        reply_markup=back_keyboard("quiz:resume"),
    )
    context.user_data["awaiting_appeal"] = True
    return

```

```

async def _start_quiz_flow(context, user_id: int, chat_id: int):
    """Показывает список тем для выбора."""
    is_prem = await db_is_premium(user_id)

    # Закрытый режим
    closed = await db_get_setting("closed_mode", "0") == "1"
    if closed:
        async with aioredis.connect(DB_PATH) as db:
            async with db.execute(
                "SELECT user_id FROM closed_access WHERE user_id=?", (user_id,)
            ) as cur:
                has_access = await cur.fetchone()
        if not has_access:
            await safe_send(
                context.bot, chat_id,
                "🔒 Бот работает в закрытом режиме.\n"
                "Доступ предоставляется администратором."
            )
            return

    topics = await db_get_topics(premium_ok=is_prem)
    if not topics:
        await safe_send(context.bot, chat_id, "🇷🇺 Темы ещё не добавлены. Скоро пс
        return

    buttons = []
    for t in topics:
        cnt = await db_count_questions(t["id"])
        icon = "💎" if t["is_premium"] else "📖"
        buttons.append([
            InlineKeyboardButton(
                f"{icon} {t['name']} [{cnt}]",
                callback_data=f"topic:{t['id']}"
            )
        ])
    buttons.append([InlineKeyboardButton("⬅️ Назад", callback_data="main:menu")])

    await safe_send(
        context.bot, chat_id,
        "📖 <b>Выберите тему для тестирования:</b>",
        parse_mode=ParseMode.HTML,
        reply_markup=InlineKeyboardMarkup(buttons),
    )

async def _cb_topic_select(query, context, user, chat_id, data):
    """Пользователь выбрал тему."""

```

```

topic_id = int(data.split(":")[1])
user_id = user.id

is_prem = await db_is_premium(user_id)
topic = None
async with aiosqlite.connect(DB_PATH) as db:
    async with db.execute("SELECT * FROM topics WHERE id=?", (topic_id,)) as cur:
        row = await cur.fetchone()
        if row:
            cols = [d[0] for d in cur.description]
            topic = dict(zip(cols, row))

if not topic:
    await query.answer("❌ Тема не найдена.", show_alert=True)
    return

if topic["is_premium"] and not is_prem:
    await query.answer("💎 Эта тема доступна только Premium-пользователям.",
    await safe_send(
        context.bot, chat_id,
        "💎 Эта тема доступна только для Premium.",
        reply_markup=InlineKeyboardMarkup([[
            InlineKeyboardButton("💎 Купить Premium", callback_data="premium:
        ]])
    )
    return

cnt = await db_count_questions(topic_id)
if cnt == 0:
    await query.answer("🇷🇺 В этой теме пока нет вопросов.", show_alert=True)
    return

# Создаём/сбрасываем сессию
await db_upsert_session(
    user_id,
    topic_id=topic_id,
    question_id=None,
    correct_count=0,
    wrong_count=0,
    total_asked=0,
    daily_count=0,
    coins_earned=0,
    xp_earned=0,
    session_date=datetime.utcnow().strftime("%Y-%m-%d"),
    status="active",
    missed_polls=0,
    reminder_sent=0,

```

```

        chat_id=chat_id,
    )

    await safe_send(
        context.bot, chat_id,
        f"🚀 <b>Тест начинается!</b>\nТема: <b>{escape(topic['name'])}</b>\n\nПер
        parse_mode=ParseMode.HTML,
    )

    await asyncio.sleep(1)
    await send_next_question(context, user_id, chat_id)

async def _cb_premium(query, context, user, chat_id, data):
    user_id = user.id

    if data == "premium:shop":
        await show_premium_shop(context.bot, chat_id, user_id)
        return

    if data.startswith("buy_prem:coins:"):
        tier = data.split(":")[-1]
        await handle_buy_premium_coins(context.bot, user_id, chat_id, tier)
        return

    if data.startswith("buy_prem:money:"):
        days = int(data.split(":")[-1])
        await handle_buy_premium_money(context.bot, user_id, chat_id, days)
        return

async def _cb_profile(query, context, user, chat_id, data):
    user_id = user.id

    if data == "profile:stats":
        user_db = await db_get_user(user_id)
        if not user_db:
            await query.answer("❌ Профиль не найден.", show_alert=True)
            return

    total_correct = 0
    total_wrong = 0
    async with aioredislite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT SUM(is_correct), COUNT(*) FROM user_answers WHERE user_id
            (user_id,)"
        ) as cur:
            row = await cur.fetchone()

```

```

        if row:
            total_correct = row[0] or 0
            total_wrong = (row[1] or 0) - total_correct

    weak = await db_get_weak_topics(user_id, 3)
    strong = await db_get_strong_topics(user_id, 3)
    total = total_correct + total_wrong
    pct = round(total_correct / max(total, 1) * 100)

    text = (
        📊 <b>Ваша статистика</b>\n\n
        f"✅ Всего правильных: {total_correct}\n"
        f"❌ Всего ошибок: {total_wrong}\n"
        f"📈 Общая точность: {pct}%\n\n"
        "💪 Сильные темы:\n" +
        (" \n".join(f" ✅ {escape(t)}" for t in strong) or " нет данных") +
        "\n\n ⚠️ Слабые темы:\n" +
        (" \n".join(f" ❌ {escape(t)}" for t in weak) or " нет данных")
    )
    await safe_send(context.bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=back_keyboard("main:menu"))
    return

if data == "profile:rating":
    board = await db_get_leaderboard(limit=10)
    lines = []
    medals = ["🥇", "🥈", "🥉"]
    for i, entry in enumerate(board):
        medal = medals[i] if i < 3 else f"{i+1}."
        name = escape(entry.get("username") or entry.get("first_name") or "")
        lines.append(f"{medal} @{name} — {entry['cnt']:,} XP")

    rank, total = await db_get_user_rank(user_id)
    text = (
        🏆 <b>Топ 10 участников</b>\n\n +
        "\n".join(lines) +
        f"\n\n 🎯 Ваше место: #{rank} из {total}"
    )
    await safe_send(context.bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=back_keyboard("main:menu"))
    return

async def _cb_referral(query, context, user, chat_id, data):
    if data == "ref:link":
        user_db = await db_get_user(user.id)
        if not user_db:

```

```

        await query.answer("❌ Профиль не найден.", show_alert=True)
        return
    bot_info = await context.bot.get_me()
    link = f"https://t.me/{bot_info.username}?start=ref_{user_db['ref_code']}"

    async with aioredis.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT COUNT(*) FROM referrals WHERE referrer_id=?", (user.id,)
        ) as cur:
            ref_count = (await cur.fetchone())[0]

    coin_per = await db_get_setting("coin_per_referral", "200")
    text = (
        "👤 <b>Реферальная программа</b>\n\n"
        f"🔗 Ваша ссылка:\n<code>{link}</code>\n\n"
        f"👥 Приглашено друзей: {ref_count}\n"
        f"🌐 Монет за каждого: {coin_per}\n\n"
        "Поделитесь ссылкой – и получите монеты за каждого нового участника!"
    )
    await safe_send(context.bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=back_keyboard("main:menu"))

async def _cb_draft(query, context, user, chat_id, data):
    """Обработка сохранения/отмены черновика."""
    if not is_admin(user.id):
        return

    parts = data.split(":")
    if len(parts) < 3:
        return

    action = parts[1]
    draft_id = int(parts[2])

    if action == "save":
        saved = await commit_draft(draft_id)
        await safe_send(context.bot, chat_id, f"✅ Сохранено {saved} вопросов.")
    elif action == "cancel":
        async with aioredis.connect(DB_PATH) as db:
            await db.execute("DELETE FROM drafts WHERE id=?", (draft_id,))
            await db.commit()
        await safe_send(context.bot, chat_id, "❌ Черновик удалён.")

async def _cb_admin(query, context, user, chat_id, data):
    """Обработка всех adm: callbacks."""

```

```

if not is_admin(user.id):
    await query.answer("🚫 Только для администраторов.", show_alert=True)
    return

if data == "adm:main":
    await show_admin_panel(context.bot, chat_id)
    return

if data == "adm:stats":
    await adm_show_stats(context.bot, chat_id)
    return

if data == "adm:tests":
    await adm_show_tests(context.bot, chat_id)
    return

if data == "adm:import":
    await adm_show_import_menu(context.bot, chat_id)
    return

if data == "adm:import:text":
    # Нужно выбрать тему сначала
    topics = await db_get_topics(premium_ok=True)
    if not topics:
        await safe_send(context.bot, chat_id,
            "❌ Сначала создайте тему (/admin → Тесты → Создать тему)")
        return
    buttons = [
        [InlineKeyboardButton(t["name"], callback_data=f"adm:import:topic:{t["id"]}"),
        ]
    ]
    buttons.append([InlineKeyboardButton("⬅️ Назад", callback_data="adm:import:main"),
    ])
    await safe_send(context.bot, chat_id, "📖 Выберите тему для импорта текста",
        reply_markup=InlineKeyboardMarkup(buttons))
    return

if data == "adm:import:poll":
    # Импорт пересланного Quiz Poll – сначала выбираем тему
    topics = await db_get_topics(premium_ok=True)
    if not topics:
        await safe_send(context.bot, chat_id,
            "❌ Сначала создайте тему (/admin → Тесты → Создать тему)")
        return
    buttons = [
        [InlineKeyboardButton(t["name"], callback_data=f"adm:import:topic:{t["id"]}"),
        ]
    ]

```

```

buttons.append([InlineKeyboardButton("⬅️ Назад", callback_data="adm:import"),
await safe_send(
    context.bot, chat_id,
    "📝 Выберите тему для импорта Quiz Poll:",
    reply_markup=InlineKeyboardMarkup(buttons),
)
return

if data.startswith("adm:import:topic:"):
    # Формат: adm:import:topic:<topic_id>:<mode>
    parts_import = data.split(":")
    topic_id = int(parts_import[3])
    import_mode = parts_import[4] if len(parts_import) > 4 else "text"

    context.user_data["admin_import_topic_id"] = topic_id
    context.user_data["admin_import_mode"] = import_mode

    if import_mode == "text":
        await safe_send(
            context.bot, chat_id,
            "✎ <b>Вставьте вопросы в формате:</b>\n\n"
            "<code>1. Вопрос?\nA) Ответ*\nB) Ответ\nC) Ответ\nD) Ответ</code>"
            "Звёздочка (*) обозначает правильный ответ.\n"
            "Можно вставить несколько вопросов сразу.",
            parse_mode=ParseMode.HTML,
            reply_markup=back_keyboard("adm:import"),
        )
    else:
        # Режим импорта Quiz Poll
        await safe_send(
            context.bot, chat_id,
            "📊 <b>Импорт Quiz Poll</b>\n\n"
            "Перешлите боту Quiz Poll из любого канала или чата.\n\n"
            "✅ Бот автоматически распознает:\n"
            " • Текст вопроса\n"
            " • Все варианты ответов\n"
            " • Правильный ответ\n\n"
            "Можно пересылать <b>несколько Quiz Poll подряд</b> – "
            "они накапливаются в черновик.\n\n"
            "После пересылки всех вопросов нажмите:\n"
            "<b>✅ Сохранить черновик</b>",
            parse_mode=ParseMode.HTML,
            reply_markup=InlineKeyboardMarkup([
                InlineKeyboardButton(
                    "✅ Сохранить черновик",
                    callback_data=f"adm:import:poll:save:{topic_id}"
                ),
            ]),

```

```

        [InlineKeyboardButton("❌ Отменить", callback_data="adm:import
    ]),
    )
    # Инициализируем буфер накопления вопросов из Poll
    context.user_data["poll_import_buffer"] = []
    return

# Сохранение накопленного Poll-буфера в draft
if data.startswith("adm:import:poll:save:"):
    topic_id = int(data.split(":")[-1])
    buffer = context.user_data.pop("poll_import_buffer", [])
    context.user_data.pop("admin_import_mode", None)
    context.user_data.pop("admin_import_topic_id", None)

    if not buffer:
        await safe_send(context.bot, chat_id,
            "❌ Вы не переслали ни одного Quiz Poll.\nПопробуйте снова.")
        return

    draft_id = await save_draft(user.id, buffer, topic_id)
    preview = "\n\n".join(
        f" ? {escape(q['text'])}\n"
        + "\n".join(
            f" {'✅' if i == q['correct'] else ' '} {chr(65+i)} {escape(o)}
            for i, o in enumerate(q['opts'])
        )
        for q in buffer[:3]
    )
    tail = f"\n\n...и ещё {len(buffer)-3}" if len(buffer) > 3 else ""
    await safe_send(
        context.bot, chat_id,
        f"📄 <b>Предпросмотр ({len(buffer)} вопросов из Poll)</b>\n\n{preview
        parse_mode=ParseMode.HTML,
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("✅ Сохранить в базу", callback_data=f"draf
            InlineKeyboardButton("❌ Отменить", callback_data=f"draf
        ]),
    )
    return

if data == "adm:users":
    await adm_show_users(context.bot, chat_id, user.id)
    return

if data == "adm:users:find":
    context.user_data["admin_find_user"] = True
    await safe_send(context.bot, chat_id,

```

```

        "🔍 Введите Telegram ID или @username пользователя:",
        reply_markup=back_keyboard("adm:users"),
    )
    return

if data == "adm:users:top":
    board = await db_get_leaderboard(limit=10)
    lines = [f"{i+1}. @{escape(e.get('username') or e.get('first_name') or '-')}
              for i, e in enumerate(board)]
    await safe_send(context.bot, chat_id,
        "🏆 <b>Топ 10 по XP</b>\n\n" + "\n".join(lines),
        parse_mode=ParseMode.HTML,
        reply_markup=back_keyboard("adm:users"),
    )
    return

if data == "adm:premium":
    async with aisqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT COUNT(*) FROM premium WHERE expires_at > ?",
            (datetime.utcnow().isoformat(),)
        ) as cur:
            active_prem = (await cur.fetchone())[0]
    await safe_send(
        context.bot, chat_id,
        f"💎 <b>Premium</b>\n\nАктивных Premium: {active_prem}\n\n"
        "Выдать Premium: введите ID через /give_premium <ID> <дни>",
        parse_mode=ParseMode.HTML,
        reply_markup=back_keyboard("adm:main"),
    )
    return

if data == "adm:economy":
    await adm_show_economy(context.bot, chat_id)
    return

if data == "adm:eco:toggle_x2":
    current = await db_get_setting("x2_coins_active", "0")
    new_val = "0" if current == "1" else "1"
    await db_set_setting("x2_coins_active", new_val)
    status = "🟢 ВКЛ" if new_val == "1" else "🔴 ВЫКЛ"
    await query.answer(f"x2 монеты: {status}", show_alert=True)
    await adm_show_economy(context.bot, chat_id)
    return

if data == "adm:eco:toggle_prem_coins":
    current = await db_get_setting("premium_coins_enabled", "1")

```

```

new_val = "0" if current == "1" else "1"
await db_set_setting("premium_coins_enabled", new_val)
status = "🟢 ВКЛ" if new_val == "1" else "🔴 ВЫКЛ"
await query.answer(f"Premium за монеты: {status}", show_alert=True)
await adm_show_economy(context.bot, chat_id)
return

if data == "adm:settings":
    await adm_show_settings(context.bot, chat_id)
    return

if data == "adm:settings:toggle_closed":
    current = await db_get_setting("closed_mode", "0")
    new_val = "0" if current == "1" else "1"
    await db_set_setting("closed_mode", new_val)
    status = "🟢 ВКЛ" if new_val == "1" else "🔴 ВЫКЛ"
    await query.answer(f"Закрытый режим: {status}", show_alert=True)
    await adm_show_settings(context.bot, chat_id)
    return

if data == "adm:settings:toggle_antifarm":
    current = await db_get_setting("antifarm_enabled", "1")
    new_val = "0" if current == "1" else "1"
    await db_set_setting("antifarm_enabled", new_val)
    status = "🟢 ВКЛ" if new_val == "1" else "🔴 ВЫКЛ"
    await query.answer(f"Антифарм: {status}", show_alert=True)
    await adm_show_settings(context.bot, chat_id)
    return

if data == "adm:appeals":
    await adm_show_appeals(context.bot, chat_id)
    return

if data.startswith("adm:appeal:"):
    parts = data.split(":")
    if len(parts) == 3:
        # adm:appeal:ID - открыть апелляцию
        await adm_show_appeal(context.bot, chat_id, int(parts[2]))
        return
    action = parts[3] if len(parts) > 3 else ""
    appeal_id = int(parts[2]) if len(parts) > 2 else 0

    if action == "accept":
        async with aioredis.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE appeals SET status='accepted' WHERE id=?", (appeal_id
            )

```

```

        await db.commit()
        await query.answer("✅ Апелляция принята.", show_alert=True)
        await adm_show_appeals(context.bot, chat_id)
        return

if action == "reject":
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            "UPDATE appeals SET status='rejected' WHERE id=?", (appeal_id,
        )
        await db.commit()
        await query.answer("❌ Апелляция отклонена.", show_alert=True)
        await adm_show_appeals(context.bot, chat_id)
        return

if action == "del_q":
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT question_id FROM appeals WHERE id=?", (appeal_id,)
        ) as cur:
            row = await cur.fetchone()
            if row:
                await db.execute(
                    "UPDATE questions SET is_active=0 WHERE id=?", (row[0],)
                )
                await db.execute(
                    "UPDATE appeals SET status='accepted', admin_note='вопрос уда
                    (appeal_id,)
                )
                await db.commit()
                await query.answer("🗑 Вопрос удалён.", show_alert=True)
                await adm_show_appeals(context.bot, chat_id)
                return

if data == "adm:broadcast":
    context.user_data["admin_broadcast_mode"] = "all"
    await safe_send(
        context.bot, chat_id,
        "📢 <b>Рассылка</b>\n\nВыберите аудиторию:",
        parse_mode=ParseMode.HTML,
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("👥 Всем",
                                callback_data="adm:
            [InlineKeyboardButton("💎 Только Premium",
                                callback_data="adm:
            [InlineKeyboardButton("🆓 Только бесплатным", callback_data="adm:
            [InlineKeyboardButton("⬅ Назад",
                                callback_data="adm:
        ]),
    )

```

```

    return

if data.startswith("adm:bc:"):
    mode = data.split(":")[2]
    context.user_data["admin_broadcast_mode"] = mode
    context.user_data["admin_broadcast_active"] = True
    await safe_send(
        context.bot, chat_id,
        f"📢 Аудитория: <b>{mode}</b>\n\nОтправьте сообщение для рассылки:",
        parse_mode=ParseMode.HTML,
        reply_markup=back_keyboard("adm:broadcast"),
    )
    return

if data == "adm:rating":
    board = await db_get_leaderboard(limit=10)
    lines = [
        f"{i+1}. @{escape(e.get('username') or e.get('first_name') or '-')}" -
        for i, e in enumerate(board)
    ]
    await safe_send(
        context.bot, chat_id,
        "🏆 <b>Рейтинг</b>\n\n" + "\n".join(lines),
        parse_mode=ParseMode.HTML,
        reply_markup=back_keyboard("adm:main"),
    )
    return

if data == "adm:access":
    closed = await db_get_setting("closed_mode", "0")
    text = (
        "🔒 <b>Управление доступом</b>\n\n"
        f"Закрытый режим: {'🟢 ВКЛ' if closed=='1' else '🔴 ВЫКЛ'}\n\n"
        "Выдать доступ: /give_access <user_id>\n"
        "Забрать доступ: /revoke_access <user_id>"
    )
    await safe_send(context.bot, chat_id, text, parse_mode=ParseMode.HTML,
                    reply_markup=back_keyboard("adm:main"))
    return

if data.startswith("adm:user:"):
    await _adm_user_card(context.bot, chat_id, data, context)
    return

async def _adm_user_card(bot, chat_id: int, data: str, context):
    """Карточка пользователя в админке."""

```

```

parts = data.split(":")
action = parts[2] if len(parts) > 2 else ""
user_id = int(parts[3]) if len(parts) > 3 else 0

user_db = await db_get_user(user_id)
if not user_db:
    await safe_send(bot, chat_id, "❌ Пользователь не найден.")
    return

if action == "view":
    is_prem = await db_is_premium(user_id)
    rank, total = await db_get_user_rank(user_id)
    lvl, lvl_name, _, _ = get_level_info(user_db["xp"])
    is_banned = user_id in BLOCKED_USERS

    text = (
        "👤 <b>Карточка пользователя</b>\n\n"
        f"ID: <code>{user_id}</code>\n"
        f"Username: @{escape(user_db.get('username') or '-')}\n"
        f"Имя: {escape(user_db.get('first_name') or '-')}\n"
        f"Уровень: {lvl_name}\n"
        f"XP: {user_db['xp']:,}\n"
        f"Монеты: {user_db['coins']:,}\n"
        f"Streak: {user_db['streak']}\n"
        f"Premium: {'✅' if is_prem else '❌'}\n"
        f"Место: #{rank} из {total}\n"
        f"Забанен: {'🚫' if is_banned else '✅'}"
    )

    buttons = InlineKeyboardMarkup([
        [InlineKeyboardButton("💎 Выдать Premium 30д", callback_data=f"adm:u
            InlineKeyboardButton("❌ Снять Premium", callback_data=f"adm:u
        [InlineKeyboardButton("🚫 Бан", callback_data=f"adm:u
            InlineKeyboardButton("✅ Разбан", callback_data=f"adm:u
        [InlineKeyboardButton("🌐 +500 монет", callback_data=f"adm:u
        [InlineKeyboardButton("🔙 Назад", callback_data="adm:us
    ])

    await safe_send(bot, chat_id, text, parse_mode=ParseMode.HTML, reply_mark
    return

if action == "prem":
    await db_grant_premium(user_id, 30, source="admin")
    await safe_send(bot, chat_id, f"✅ Premium 30 дней выдан пользователю {us
    return

if action == "revprem":
    await db_revoke_premium(user_id)
    await safe_send(bot, chat_id, f"❌ Premium снят у пользователя {user_id}")

```

```

        return

    if action == "ban":
        BLOCKED_USERS.add(user_id)
        async with aioredis.connect(DB_PATH) as db:
            await db.execute("UPDATE users SET is_banned=1 WHERE id=?", (user_id,))
            await db.commit()
        await safe_send(bot, chat_id, f"🚫 Пользователь {user_id} заблокирован.")
        return

    if action == "unban":
        BLOCKED_USERS.discard(user_id)
        async with aioredis.connect(DB_PATH) as db:
            await db.execute("UPDATE users SET is_banned=0 WHERE id=?", (user_id,))
            await db.commit()
        await safe_send(bot, chat_id, f"✅ Пользователь {user_id} разблокирован.")
        return

    if action == "addcoins":
        await db_add_coins(user_id, 500, "admin_gift")
        await safe_send(bot, chat_id, f"🌐 Пользователю {user_id} добавлено 500 м")
        return

# — MAIN MESSAGE HANDLER —————

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Единая точка входа для всех сообщений (не команд)."""
    message = update.message
    user = update.effective_user
    if not message or not user:
        return

    # Антиспам
    if not check_rate_limit(user.id, max_msgs=15, window=10):
        return

    # — Ответ администратора на обращение —————
    if is_admin(user.id):
        await _handle_admin_message(update, context)
        return

    # — Обычный пользователь заблокирован —————
    if user.id in BLOCKED_USERS:
        warn = await message.reply_text(
            f"{safe_username(user)}, вы заблокированы администрацией."
        )

```

```

        asyncio.create_task(delete_message_later(context, warn.chat_id, warn.mess
        return

# — Апелляция —————
if context.user_data.get("awaiting_appeal"):
    await _handle_appeal_text(update, context)
    return

# — Диалог завершён —————
if user.id in FINISHED_USERS:
    warn = await message.reply_text(
        "Диалог завершён.\nДля нового вопроса нажмите /start"
    )
    asyncio.create_task(delete_message_later(context, warn.chat_id, warn.mess
    return

# — Обращение (A/B/C/D) —————
started = context.user_data.get("started", False)
reason_selected = context.user_data.get("reason_selected", False)
reason_code = context.user_data.get("reason_code")
reason_title = context.user_data.get("reason_title")

if not started:
    warn = await message.reply_text("⚠️ Нажмите /start")
    asyncio.create_task(delete_message_later(context, warn.chat_id, warn.mess
    return

if not reason_selected or not reason_code:
    warn = await message.reply_text(
        "⚠️ Сначала выберите причину обращения.",
        reply_markup=reason_keyboard(),
    )
    asyncio.create_task(delete_message_later(context, warn.chat_id, warn.mess
    return

await _handle_user_request(update, context, reason_code, reason_title)

async def _handle_appeal_text(update: Update, context: ContextTypes.DEFAULT_TYPE)
    """Пользователь ввёл текст апелляции."""
    message = update.message
    user = update.effective_user
    question_id = context.user_data.get("appeal_question_id")

    if not question_id:
        await message.reply_text("❌ Ошибка. Попробуйте снова.")
        context.user_data["awaiting_appeal"] = False

```

```

return

text = message.text or ""
parts = text.split("|")
comment = parts[0].strip() if parts else text
source_ref = parts[1].strip() if len(parts) > 1 else ""

async with aioredislite.connect(DB_PATH) as db:
    await db.execute("""
        INSERT INTO appeals(user_id, question_id, comment, source_ref)
        VALUES(?,?,?,?)
        """, (user.id, question_id, comment, source_ref))
    await db.commit()

context.user_data["awaiting_appeal"] = False

question = await db_get_question_by_id(question_id)
if question:
    opts = [
        question["option_a"], question["option_b"],
        question["option_c"], question["option_d"]
    ]
    opts_text = "\n".join(
        f" {img} if i == question['correct_index'] else ' ' {chr(65+i))} {"
        for i, o in enumerate(opts)
    )
    for adm_id in ADMINS:
        await safe_send(
            context.bot, adm_id,
            f"<b>#АПЕЛЛЯЦИЯ</b>\n\n"
            f"ID: <code>{question['qid']}</code>\n"
            f"@{escape(user.username or str(user.id))}\n"
            f"<img alt='book icon'> Тема ID: {question['topic_id']}\n"
            f"? {escape(question['question_text'])}\n\n"
            f"{opts_text}\n\n"
            f"<img alt='speech bubble icon'> Комментарий: {escape(comment)}\n"
            f"<img alt='book icon'> Источник: {escape(source_ref)}",
            parse_mode=ParseMode.HTML,
        )

await message.reply_text(
    "✔️ Апелляция отправлена! Тест продолжается.",
    reply_markup=pause_keyboard(),
)

# Возобновляем тест
session = await db_get_session(user.id)
if session:

```

```

        await db_upsert_session(user.id, status="active")
        await send_next_question(context, user.id, update.effective_chat.id)

async def _handle_user_request(update: Update, context: ContextTypes.DEFAULT_TYPE
                                reason_code: str, reason_title: str):
    """Старая система обращений (A/B/C/D)."""
    message = update.message
    user = update.effective_user
    chat_id = update.effective_chat.id

    request_id = next(REQUEST_SEQ)
    msg_type = detect_message_type(message)

    REQUESTS[request_id] = {
        "reason_code": reason_code,
        "reason_title": reason_title,
        "status": "open",
        "status_text": "НЕ ОТБЕЧЕНО ❌",
        "answered_by": None,
        "user_reaction": None,
        "admin_reactions": {"👍": set(), "👎": set()},
        "user": {
            "id": user.id, "username": user.username,
            "first_name": user.first_name, "last_name": user.last_name,
        },
        "message_type": msg_type,
        "message_text": message.text if message.text else None,
        "caption": message.caption if message.caption else None,
        "voice_duration": message.voice.duration if message.voice else None,
        "admin_message_refs": [],
    }

    admin_text = build_admin_card_text(request_id)
    admin_markup = admin_card_keyboard(request_id)

    for admin_id in ADMINS:
        try:
            sent = await context.bot.send_message(
                chat_id=admin_id, text=admin_text,
                parse_mode=ParseMode.HTML, reply_markup=admin_markup,
                disable_web_page_preview=True,
            )
            REQUESTS[request_id]["admin_message_refs"].append({
                "chat_id": admin_id, "message_id": sent.message_id,
            })
        if not message.text:

```

```

        await message.forward(chat_id=admin_id)
    except Exception as e:
        logger.warning(f"Не удалось отправить обращение админу {admin_id}: {e}")

    ok = await message.reply_text("✅ Сообщение отправлено администрации.")
    asyncio.create_task(delete_message_later(context, ok.chat_id, ok.message_id,

async def _handle_admin_message(update: Update, context: ContextTypes.DEFAULT_TYP
    """Все входящие сообщения от администратора."""
    message = update.message
    admin = update.effective_user
    chat_id = update.effective_chat.id

    # — Импорт пересланного Quiz Poll —————
    if context.user_data.get("admin_import_mode") == "poll":
        topic_id = context.user_data.get("admin_import_topic_id")
        if not topic_id:
            await message.reply_text("❌ Тема не выбрана. Начните заново через /a
            context.user_data.pop("admin_import_mode", None)
            return

        # Telegram передаёт пересланный Poll через message.poll
        poll = message.poll
        if not poll:
            await message.reply_text(
                "⚠️ Это не Quiz Poll.\n\n"
                "Перешлите Quiz Poll из канала/чата.\n"
                "Когда закончите — нажмите <b>✅ Сохранить черновик</b>",
                parse_mode=ParseMode.HTML,
            )
            return

        # Проверяем что это именно Quiz (не обычный опрос)
        if poll.type != "quiz":
            await message.reply_text(
                "⚠️ Это обычный Poll, не Quiz.\n"
                "Нужен именно Quiz Poll (с одним правильным ответом).\"
            )
            return

        # Парсим данные из Poll
        question_text = poll.question
        options = [opt.text for opt in poll.options]
        correct_index = poll.correct_option_id # 0-3

        # Telegram иногда не раскрывает correct_option_id для чужих poll

```

```

# В таком случае помечаем 0 и сообщаем об этом
if correct_index is None:
    correct_index = 0
    await message.reply_text(
        "⚠ Не удалось определить правильный ответ автоматически.\n"
        "Telegram скрывает его для чужих Poll.\n"
        "Вопрос добавлен с ответом А — отредактируйте через /admin → Тест
    )

if len(options) != 4:
    await message.reply_text(
        f"⚠ Вопрос пропущен: нужно ровно 4 варианта ответа, "
        f"найдено {len(options)}.\n\n"
        f"? {escape(question_text)}",
        parse_mode=ParseMode.HTML,
    )
    return

# Добавляем в буфер
buffer = context.user_data.get("poll_import_buffer", [])
buffer.append({
    "text": question_text,
    "opts": options,
    "correct": correct_index,
})
context.user_data["poll_import_buffer"] = buffer
count_now = len(buffer)

# Показываем подтверждение добавления
opts_preview = "\n".join(
    f" {'✅' if i == correct_index else ' '} {chr(65+i)} {escape(o)}"
    for i, o in enumerate(options)
)

await message.reply_text(
    f"✅ <b>Добавлен вопрос #{count_now}</b>\n\n"
    f"? {escape(question_text)}\n"
    f"{opts_preview}\n\n"
    f"📝 В черновике: {count_now} вопросов\n\n"
    "Пересылайте следующий Quiz Poll или нажмите <b>✅ Сохранить черновик"
    parse_mode=ParseMode.HTML,
    reply_markup=InlineKeyboardMarkup([
        [InlineKeyboardButton(
            f"✅ Сохранить черновик ({count_now} вопр.)",
            callback_data=f"adm:import:poll:save:{topic_id}"
        )],
        [InlineKeyboardButton("❌ Отменить", callback_data="adm:import")]
    ]),

```

```

    )
    return

# — Импорт вопросов текстом —————
if context.user_data.get("admin_import_mode") == "text":
    topic_id = context.user_data.get("admin_import_topic_id")
    if not topic_id:
        await message.reply_text("❌ Тема не выбрана. Начните заново через /a
        return

    raw_text = message.text or message.caption or ""
    if not raw_text:
        await message.reply_text("❌ Пришлите текст с вопросами.")
        return

    questions = parse_questions_text(raw_text)
    if not questions:
        await message.reply_text(
            "❌ Не удалось распознать вопросы.\n\n"
            "Формат:\n1. Вопрос?\nA) Ответ*\nB) Ответ\nC) Ответ\nD) Ответ"
        )
        return

    draft_id = await save_draft(admin.id, questions, topic_id)
    preview = "\n\n".join(
        f" ? {escape(q['text'])}\n"
        + "\n".join(
            f" {'✅' if i == q['correct'] else ' '} {chr(65+i)} {escape(o)}
            for i, o in enumerate(q["opts"])
        )
        for q in questions[:3]
    )
    await message.reply_text(
        f"📄 <b>Предпросмотр ({len(questions)} вопросов)</b>\n\n{preview}\n\n
        + (f"...и ещё {len(questions)-3}" if len(questions) > 3 else ""),
        parse_mode=ParseMode.HTML,
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("✅ Сохранить", callback_data=f"draft:sav
            InlineKeyboardButton("❌ Отменить", callback_data=f"draft:can
        ]),
    )
    context.user_data.pop("admin_import_mode", None)
    return

# Поиск пользователя
if context.user_data.get("admin_find_user"):
    context.user_data.pop("admin_find_user", None)

```

```

query_text = message.text.strip() if message.text else ""
found_user = None

if query_text.isdigit():
    found_user = await db_get_user(int(query_text))
elif query_text.startswith("@"):
    async with aisqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT * FROM users WHERE username=?", (query_text[1:],)
        ) as cur:
            row = await cur.fetchone()
            if row:
                cols = [d[0] for d in cur.description]
                found_user = dict(zip(cols, row))

if not found_user:
    await message.reply_text("❌ Пользователь не найден.")
    return

uid = found_user["id"]
await safe_send(
    context.bot, chat_id,
    f"✅ Найден: {found_user.get('first_name')} @{found_user.get('username')}  

    reply_markup=InlineKeyboardMarkup([[
        InlineKeyboardButton("👤 Открыть карточку", callback_data=f"adm:u
    ])],
)
return

```

Рассылка

```

if context.user_data.get("admin_broadcast_active"):
    context.user_data.pop("admin_broadcast_active", None)
    mode = context.user_data.pop("admin_broadcast_mode", "all")

# Получаем список user_id
async with aisqlite.connect(DB_PATH) as db:
    if mode == "premium":
        async with db.execute(
            "SELECT user_id FROM premium WHERE expires_at > ?",
            (datetime.utcnow().isoformat(),)
        ) as cur:
            rows = await cur.fetchall()
            target_ids = [r[0] for r in rows]
    elif mode == "free":
        async with db.execute("""
            SELECT u.id FROM users u
            LEFT JOIN premium p ON p.user_id=u.id

```

```

        WHERE p.user_id IS NULL OR p.expires_at < ?
        """, (datetime.utcnow().isoformat(),)) as cur:
            rows = await cur.fetchall()
            target_ids = [r[0] for r in rows]
    else:
        async with db.execute("SELECT id FROM users") as cur:
            rows = await cur.fetchall()
            target_ids = [r[0] for r in rows]

    sent_ok = 0
    sent_err = 0
    for tid in target_ids:
        try:
            await context.bot.copy_message(
                chat_id=tid,
                from_chat_id=chat_id,
                message_id=message.message_id,
            )
            sent_ok += 1
        except Forbidden:
            sent_err += 1
        except Exception:
            sent_err += 1
        await asyncio.sleep(0.05) # Rate limit

    await message.reply_text(
        f"📧 Рассылка завершена.\n"
        f"✅ Отправлено: {sent_ok}\n"
        f"❌ Ошибок: {sent_err}"
    )
    return

# Ответ на обращение (старая система A/B/C/D)
request_id = context.user_data.get("reply_request_id")
if not request_id:
    return

if request_id not in REQUESTS:
    context.user_data.pop("reply_request_id", None)
    await message.reply_text("❌ Обращение не найдено.")
    return

req = REQUESTS[request_id]
target_user_id = req["user"]["id"]

if target_user_id in BLOCKED_USERS:
    warn = await message.reply_text("🚫 Пользователь заблокирован.")

```

```

        asyncio.create_task(delete_message_later(context, warn.chat_id, warn.mess
    return

text_to_send = message.text or message.caption
sent_message = None

try:
    reply_caption = (
        "📬 <b>Ответ от администрации</b>\n\n"
        f"{escape(text_to_send or '')}"
    )

    if message.text:
        sent_message = await context.bot.send_message(
            chat_id=target_user_id, text=reply_caption, parse_mode=ParseMode.
        )
    elif message.photo:
        sent_message = await context.bot.send_photo(
            chat_id=target_user_id, photo=message.photo[-1].file_id,
            caption=reply_caption, parse_mode=ParseMode.HTML
        )
    elif message.document:
        sent_message = await context.bot.send_document(
            chat_id=target_user_id, document=message.document.file_id,
            caption=reply_caption, parse_mode=ParseMode.HTML
        )
    elif message.video:
        sent_message = await context.bot.send_video(
            chat_id=target_user_id, video=message.video.file_id,
            caption=reply_caption, parse_mode=ParseMode.HTML
        )
    elif message.voice:
        sent_message = await context.bot.send_voice(
            chat_id=target_user_id, voice=message.voice.file_id,
            caption=reply_caption, parse_mode=ParseMode.HTML
        )
    else:
        sent_message = await context.bot.copy_message(
            chat_id=target_user_id, from_chat_id=chat_id,
            message_id=message.message_id,
        )

    if sent_message:
        response_id = next(RESPONSE_SEQ)
        RESPONSES[response_id] = {
            "request_id": request_id,
            "user_id": target_user_id,

```

```

        "message_id": sent_message.message_id,
        "reaction": None,
    }
    try:
        await context.bot.edit_message_reply_markup(
            chat_id=target_user_id,
            message_id=sent_message.message_id,
            reply_markup=user_response_keyboard(response_id),
        )
    except Exception:
        pass

    req["status"] = "answered"
    req["answered_by"] = admin_name(admin)
    req["status_text"] = f"ОТВЕЧЕНО 🟢, админом: {req['answered_by']}"
    await refresh_admin_cards(context, request_id)
    context.user_data.pop("reply_request_id", None)

    ok = await message.reply_text("🟢 Ответ отправлен анонимно.")
    asyncio.create_task(delete_message_later(context, ok.chat_id, ok.message_id))

except Exception as e:
    logger.exception("Ошибка при отправке ответа")
    err = await message.reply_text(f"❌ Не удалось отправить: {e}")
    asyncio.create_task(delete_message_later(context, err.chat_id, err.message_id))

# — Старые callback обработчики (A/B/C/D система) —————

async def handle_reply_button(update: Update, context: ContextTypes.DEFAULT_TYPE)
    query = update.callback_query
    admin = update.effective_user
    if not is_admin(admin.id):
        await query.answer("Только для админов", show_alert=True)
        return
    parts = (query.data or "").split(":")
    if len(parts) != 2:
        return
    request_id = int(parts[1])
    if request_id not in REQUESTS:
        await query.answer("Обращение не найдено", show_alert=True)
        return
    context.user_data["reply_request_id"] = request_id
    await query.message.reply_text(
        "👉 Режим ответа включён. Следующее сообщение уйдёт пользователю анонимно"
    )

```

```

async def handle_admin_reaction(update: Update, context: ContextTypes.DEFAULT_TYP
    query = update.callback_query
    admin = update.effective_user
    if not is_admin(admin.id):
        return
    parts = (query.data or "").split(":")
    if len(parts) != 3:
        return
    request_id = int(parts[1])
    reaction = parts[2]
    if request_id not in REQUESTS:
        return
    req = REQUESTS[request_id]
    req["admin_reactions"].setdefault("👍", set())
    req["admin_reactions"].setdefault("👎", set())
    if admin.id in req["admin_reactions"].get(reaction, set()):
        req["admin_reactions"][reaction].discard(admin.id)
    else:
        for k in req["admin_reactions"]:
            req["admin_reactions"][k].discard(admin.id)
        req["admin_reactions"][reaction].add(admin.id)
    await refresh_admin_cards(context, request_id)

```

```

async def handle_user_reaction(update: Update, context: ContextTypes.DEFAULT_TYPE
    query = update.callback_query
    user = update.effective_user
    if is_admin(user.id):
        return
    parts = (query.data or "").split(":")
    if len(parts) != 3:
        return
    response_id = int(parts[1])
    reaction = parts[2]
    if response_id not in RESPONSES:
        await query.answer("Сообщение не найдено", show_alert=True)
        return
    resp = RESPONSES[response_id]
    if resp["user_id"] != user.id:
        await query.answer("Это не ваше сообщение", show_alert=True)
        return
    if resp.get("reaction") is not None:
        await query.answer("Реакция уже поставлена", show_alert=True)
        return
    resp["reaction"] = reaction
    request_id = resp["request_id"]

```

```

if request_id in REQUESTS:
    REQUESTS[request_id]["user_reaction"] = reaction
    await refresh_admin_cards(context, request_id)
try:
    await context.bot.edit_message_reply_markup(
        chat_id=query.message.chat_id,
        message_id=query.message.message_id,
        reply_markup=user_response_keyboard(response_id, selected=reaction),
    )
except Exception:
    pass

async def handle_block_user(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    admin = update.effective_user
    if not is_admin(admin.id):
        return
    parts = (query.data or "").split(":")
    request_id = int(parts[1])
    if request_id not in REQUESTS:
        return
    req = REQUESTS[request_id]
    user_id = req["user"]["id"]
    BLOCKED_USERS.add(user_id)
    async with aioredis.connect(DB_PATH) as db:
        await db.execute("UPDATE users SET is_banned=1 WHERE id=?", (user_id,))
        await db.commit()
    req["status"] = "blocked"
    req["status_text"] = f"ЗАБЛОКИРОВАН 🚫, админом: {admin_name(admin)}"
    await refresh_admin_cards(context, request_id)
    await safe_send(context.bot, user_id, "Вы заблокированы администрацией.")

async def handle_unblock_user(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    admin = update.effective_user
    if not is_admin(admin.id):
        return
    parts = (query.data or "").split(":")
    request_id = int(parts[1])
    if request_id not in REQUESTS:
        return
    req = REQUESTS[request_id]
    user_id = req["user"]["id"]
    BLOCKED_USERS.discard(user_id)
    async with aioredis.connect(DB_PATH) as db:

```

```

        await db.execute("UPDATE users SET is_banned=0 WHERE id=?", (user_id,))
        await db.commit()
    req["status"] = "unblocked"
    req["status_text"] = f"РАЗБЛОКИРОВАН ✅, админом: {admin_name(admin)}"
    await refresh_admin_cards(context, request_id)
    await safe_send(context.bot, user_id, "✅ Вы разблокированы. Нажмите /start")

async def handle_finish_dialog(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    admin = update.effective_user
    if not is_admin(admin.id):
        return
    parts = (query.data or "").split(":")
    request_id = int(parts[1])
    if request_id not in REQUESTS:
        return
    req = REQUESTS[request_id]
    user_id = req["user"]["id"]
    FINISHED_USERS.add(user_id)
    req["status"] = "finished"
    req["status_text"] = f"ЗАБЕРИТЕ ✅, админом: {admin_name(admin)}"
    await refresh_admin_cards(context, request_id)
    await safe_send(
        context.bot, user_id,
        "Спасибо за обращение! 😊 Если появятся вопросы — нажмите /start"
    )

# — ADMIN COMMANDS —————

async def cmd_give_premium(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Команда: /give_premium <user_id> <days>"""
    if not is_admin(update.effective_user.id):
        return
    args = context.args or []
    if len(args) < 2:
        await update.message.reply_text("Использование: /give_premium <user_id> <days>")
        return
    try:
        target_id = int(args[0])
        days = int(args[1])
    except ValueError:
        await update.message.reply_text("❌ Неверные аргументы.")
        return

    await db_grant_premium(target_id, days, source="admin")

```

```

await update.message.reply_text(f"✅ Premium {days} дней выдан пользователю {
await safe_send(
    context.bot, target_id,
    f"🎉 <b>Вам выдан Premium на {days} дней!</b>",
    parse_mode=ParseMode.HTML,
)

async def cmd_give_access(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Команда: /give_access <user_id>"""
    if not is_admin(update.effective_user.id):
        return
    args = context.args or []
    if not args:
        await update.message.reply_text("Использование: /give_access <user_id>")
        return
    try:
        uid = int(args[0])
    except ValueError:
        await update.message.reply_text("❌ Неверный ID.")
        return

    async with aioredis.connect(DB_PATH) as db:
        await db.execute(
            "INSERT OR REPLACE INTO closed_access(user_id, granted_by) VALUES(?, ?"
            (uid, update.effective_user.id)
        )
        await db.commit()
    await update.message.reply_text(f"✅ Доступ выдан пользователю {uid}")

async def cmd_revoke_access(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Команда: /revoke_access <user_id>"""
    if not is_admin(update.effective_user.id):
        return
    args = context.args or []
    if not args:
        await update.message.reply_text("Использование: /revoke_access <user_id>")
        return
    try:
        uid = int(args[0])
    except ValueError:
        await update.message.reply_text("❌ Неверный ID.")
        return

    async with aioredis.connect(DB_PATH) as db:
        await db.execute("DELETE FROM closed_access WHERE user_id=?", (uid,))

```

```

        await db.commit()
        await update.message.reply_text(f"❌ Доступ отозван у пользователя {uid}")

async def cmd_add_topic(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Команда: /add_topic <название> [premium]"""
    if not is_admin(update.effective_user.id):
        return
    args = context.args or []
    if not args:
        await update.message.reply_text("Использование: /add_topic <название> [premium]")
        return
    name = " ".join(args[:-1]) if args[-1] == "premium" else " ".join(args)
    is_premium = 1 if args and args[-1] == "premium" else 0

    async with aioredis.connect(DB_PATH) as db:
        await db.execute(
            "INSERT INTO topics(name, is_premium) VALUES(?,?)", (name, is_premium)
        )
        await db.commit()
    icon = "💎" if is_premium else "🆓"
    await update.message.reply_text(f"✅ Тема создана: {icon} {name}")

# =====
# || BACKGROUND JOBS ||
# =====

async def job_check_premiums(context: ContextTypes.DEFAULT_TYPE):
    """Каждый час: проверка истёкших Premium."""
    await check_expired_premiums(context)

async def job_notify_limit_restored(context: ContextTypes.DEFAULT_TYPE):
    """Каждый день в полночь UTC: уведомления о восстановлении лимита."""
    await notify_limit_restored(context)

# =====
# || MAIN ||
# =====

def main():
    """Точка входа."""
    # Инициализация БД
    asyncio.get_event_loop().run_until_complete(db_init())
    logger.info("✅ База данных инициализирована")

```

```

app = (
    ApplicationBuilder()
        .token(BOT_TOKEN)
        .build()
)

# — Commands —————
app.add_handler(CommandHandler("start",      cmd_start))
app.add_handler(CommandHandler("admin",      cmd_admin))
app.add_handler(CommandHandler("id",         cmd_id))
app.add_handler(CommandHandler("cancel",     cmd_cancel))
app.add_handler(CommandHandler("profile",    cmd_profile))
app.add_handler(CommandHandler("banlist",    cmd_banlist))
app.add_handler(CommandHandler("give_premium", cmd_give_premium))
app.add_handler(CommandHandler("give_access", cmd_give_access))
app.add_handler(CommandHandler("revoke_access", cmd_revoke_access))
app.add_handler(CommandHandler("add_topic",  cmd_add_topic))

# — Callbacks —————
app.add_handler(CallbackQueryHandler(callback_router))

# — Poll answers —————
app.add_handler(PollAnswerHandler(handle_poll_answer))

# — Messages —————
app.add_handler(MessageHandler(filters.ALL & ~filters.COMMAND, handle_message))

# — Background jobs —————
jq: JobQueue = app.job_queue
jq.run_repeating(job_check_premiums,      interval=3600, first=60)
jq.run_repeating(job_notify_limit_restored, interval=86400, first=300)

logger.info("🚀 Бот запущен (polling)...")
app.run_polling(allowed_updates=["message", "callback_query", "poll_answer"],

if __name__ == "__main__":
    main()

```